PUC Minas

CULTURA E PRÁTICAS DEVSECOPS

Edgar Paes

PUC Minas

SECURITY SDLC: BUILD E TEST

PUC Minas

STATIC APPLICATION SECURITY TESTING

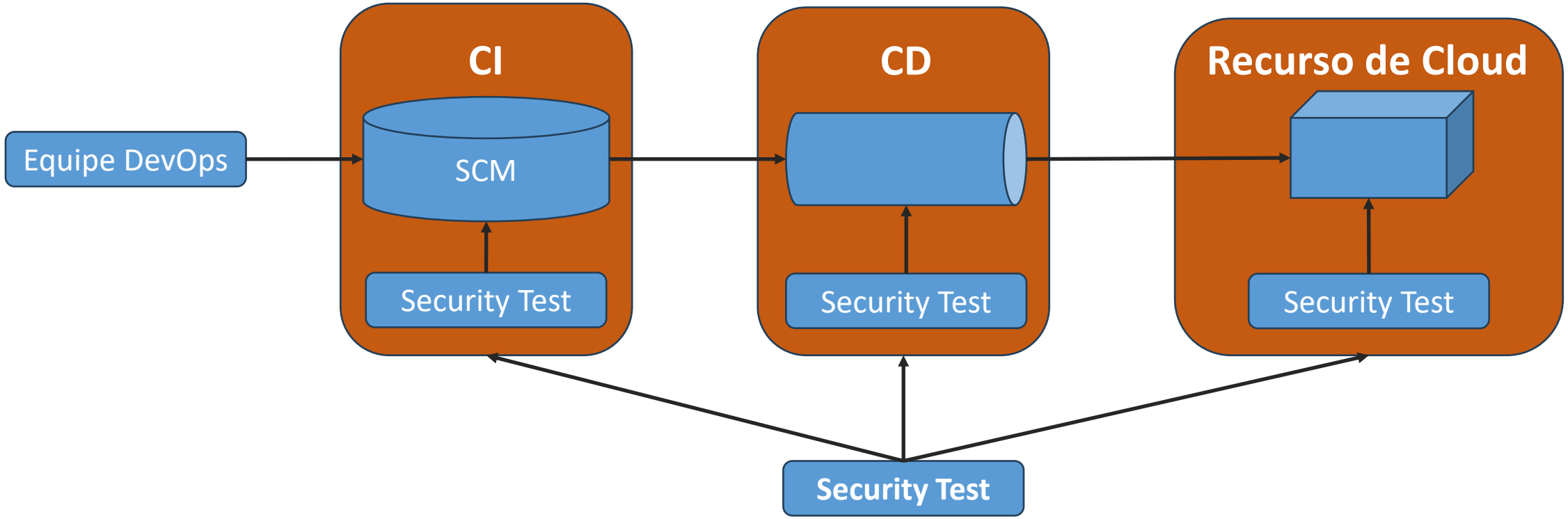
OBJETIVO

Compreender os principais conceitos primários e fundamentais que envolvem as fases de Build e Test da Segurança de Aplicações

ASPCTOS-CHAVE

- A automação de compilação (build) é crucial para os processos de DevOps, pois permite a compilação do código-fonte em código binário
- Evita erros humanos ao executar as etapas manualmente
- Automação de build e de teste permitem que os desenvolvedores verifiquem o aplicativo e as dependências de terceiros em busca de vulnerabilidades de segurança
- As principais tecnologias abordadas nestas fases incluem **SAST** e **DAST**

TEST-DRIVEN SECURITY



Na abordagem de Test-Driven Security, testes de segurança são aplicados em todo o Pipeline. Para cada vulnerabilidade encontrada nas fases, requer-se feedback imediato para a devida remediação

TEST-DRIVEN SECURITY

"Testes de software mostram a presença de bugs, mas não a sua ausência".

(Edsger W. Dijkstra)

STATIC APPLICATION SECURITY TESTING (SAST)

- Ferramentas SAST durante a fase de Build utilizam regras OWASP para avaliar vulnerabilidades no código-fonte
- O scan executa como uma tarefa do Pipeline que deve ser avaliado para não impactar no tempo de análise

Build Time-Check → **SAST**

- Integrado ao Pipeline
- Configuração de regras abrangentes
- Configuração de avaliação de SCA
- Testes automatizados baseado em risco
- Alertar partes interessadas dos riscos
- Validar assinatura digital dos artefatos de software

STATIC APPLICATION SECURITY TESTING (SAST)

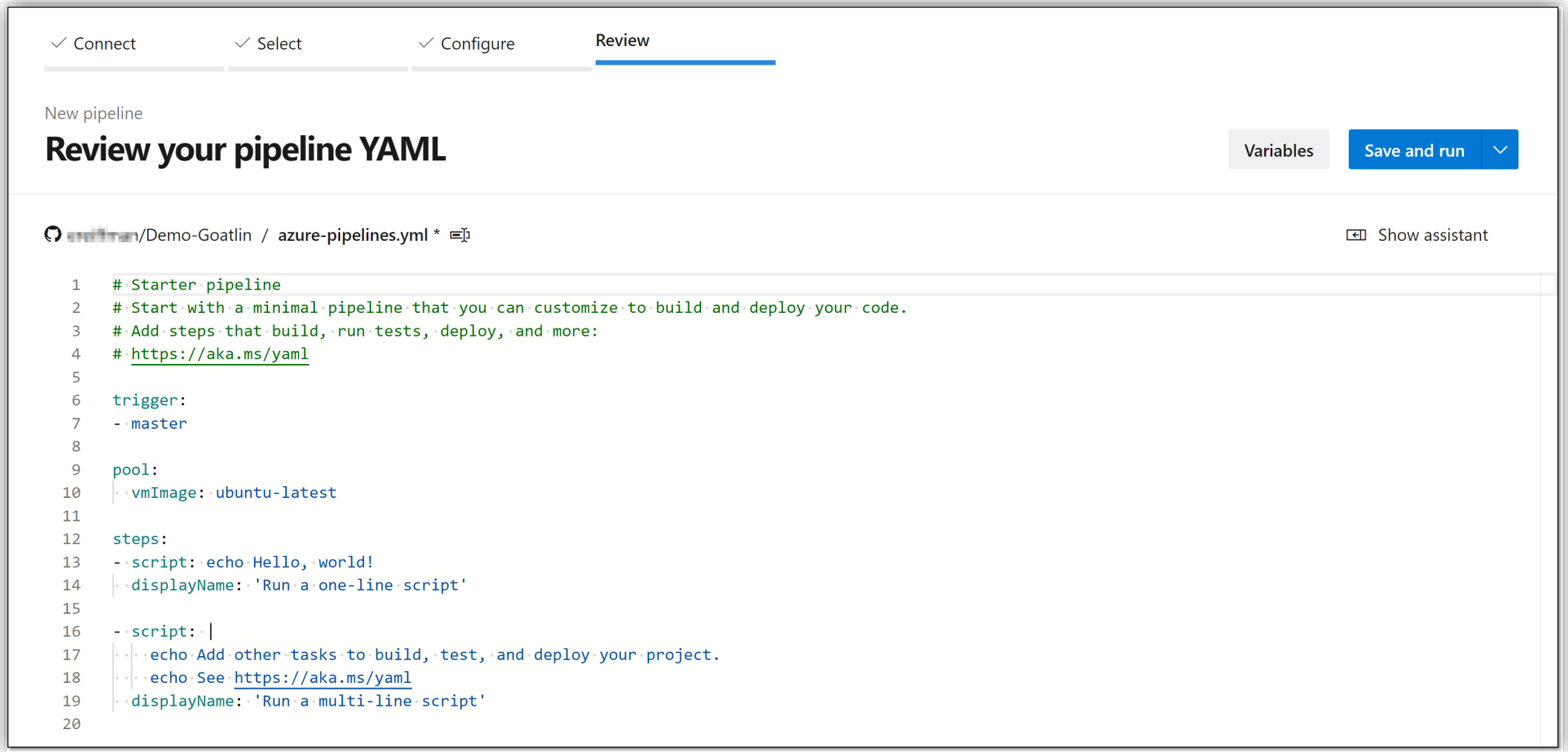
- No estágio de pré-commit, a ferramenta SAST deve verificar os objetos antes de ocorrer o commit no repositório
- No estágio de commit, a verificação SAST deve verificar todos os objetos relacionados ao código
- No estágio de compilação, a varredura SAST deve fazer parte da compilação
- No estágio de implantação, a ferramenta SAST deve ser iterativa (IAST) e deve garantir que nenhuma vulnerabilidade seja encontrada

INTEGRAÇÃO SAST AO PIPELINE (GITHUB ACTIONS)

```yaml
name: Checkmarx One Scan
on:
  push:
    branches: [ “develop" ]
  pull_request:
    branches: [ “develop" ]
  workflow_dispatch:
jobs:
  checkmarx-scan:
    name: CxOne SAST and SCA Scan
    runs-on: ubuntu-latest
```

```yaml
steps:
      - name: Checkout do código
        uses: actions/checkout@v4
      - name: Executar Checkmarx One
        uses: checkmarx/ast-github-action@v2
        with:
          base_uri: ${{ secrets.CX_BASE_URI }}
          cx_tenant: ${{ secrets.CX_TENANT }}
          cx_client_id: ${{ secrets.CX_CLIENT_ID }}
          cx_client_secret: ${{ secrets.CX_CLIENT_SECRET }}
          additional_params: --scan-types sast,sca
```

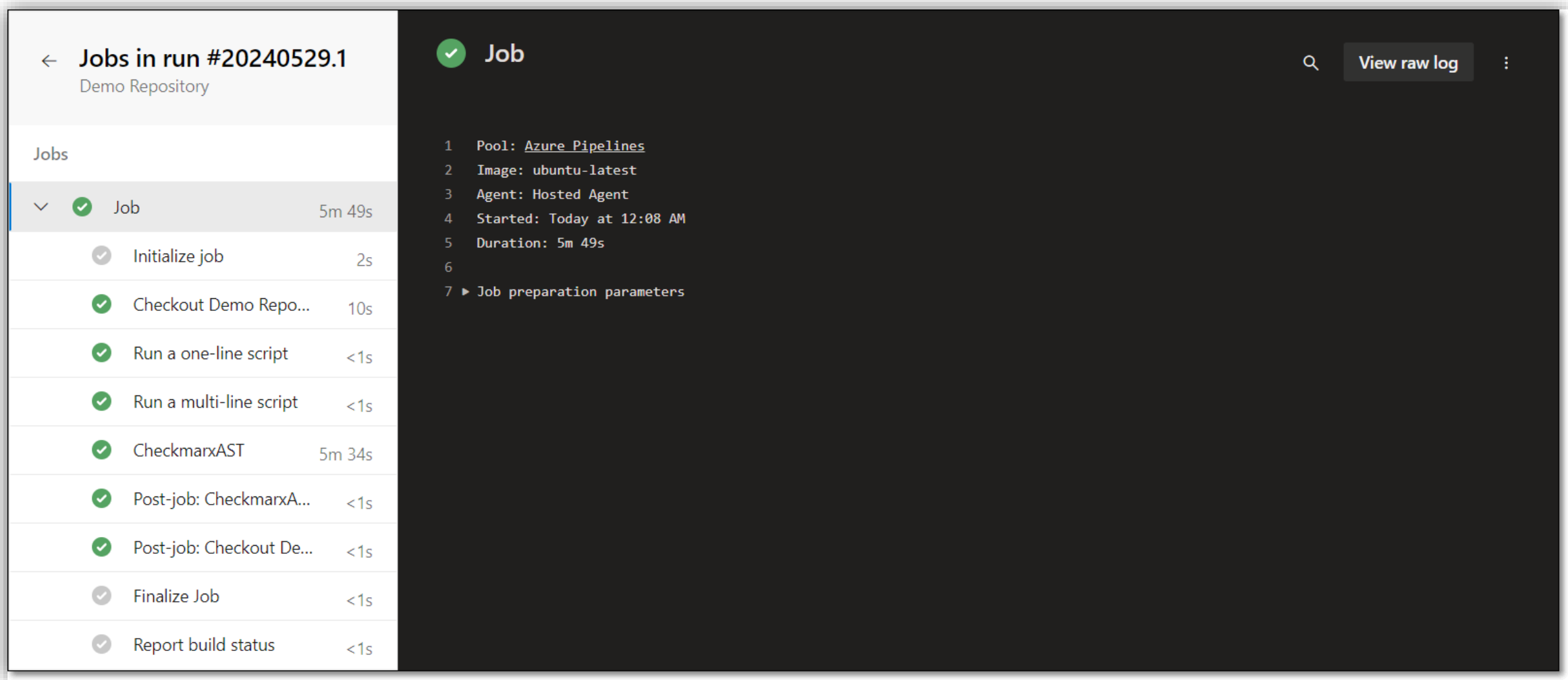
INTEGRAÇÃO SAST AO PIPELINE



Fonte: https://docs.checkmarx.com. Integração do Checkmarx One com o Azure DevOps.

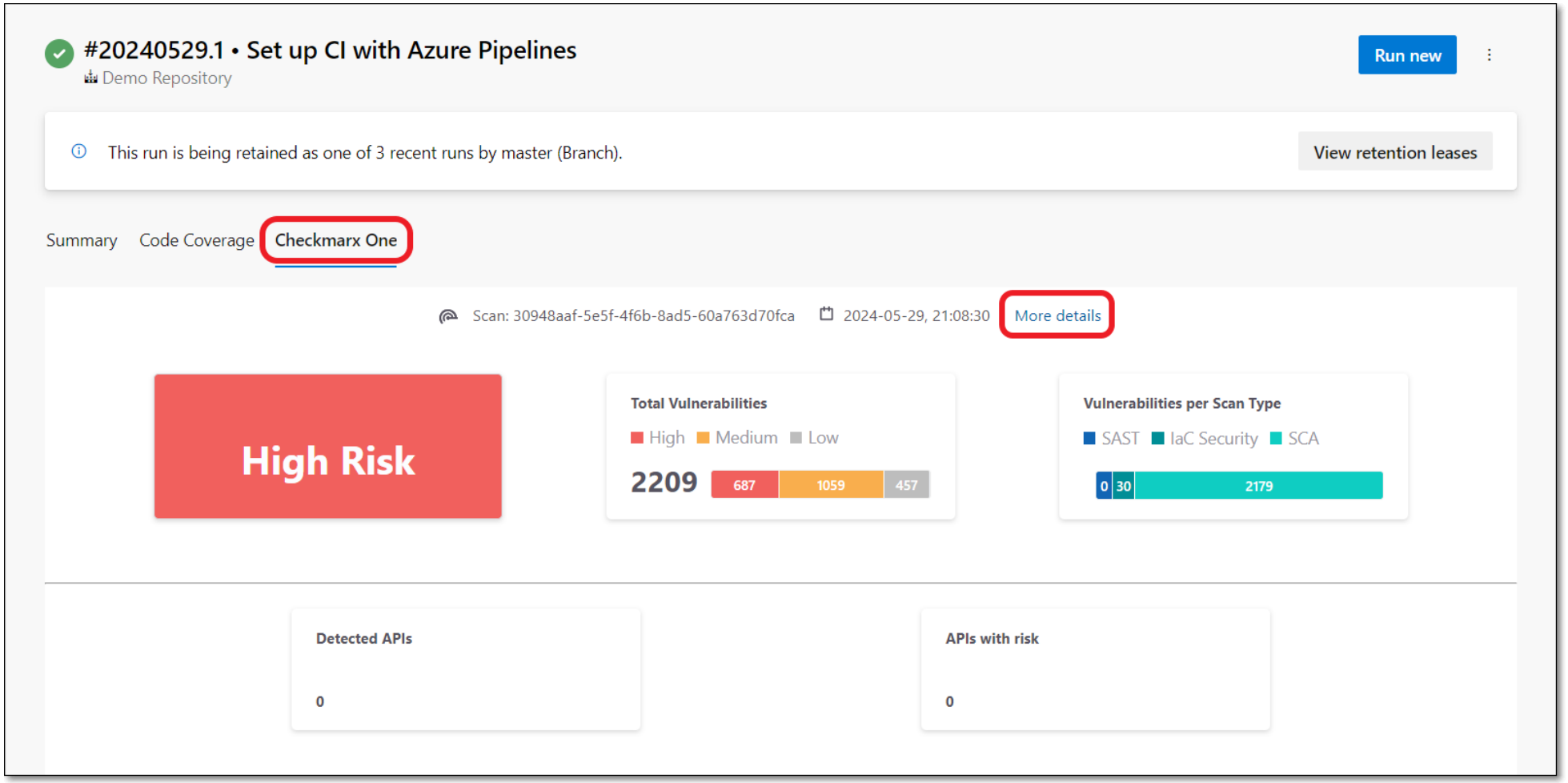
PUC Minas

INTEGRAÇÃO SAST AO PIPELINE (AZURE DEVOPS)



Fonte: https://docs.checkmarx.com. Integração do Checkmarx One com o Azure DevOps.

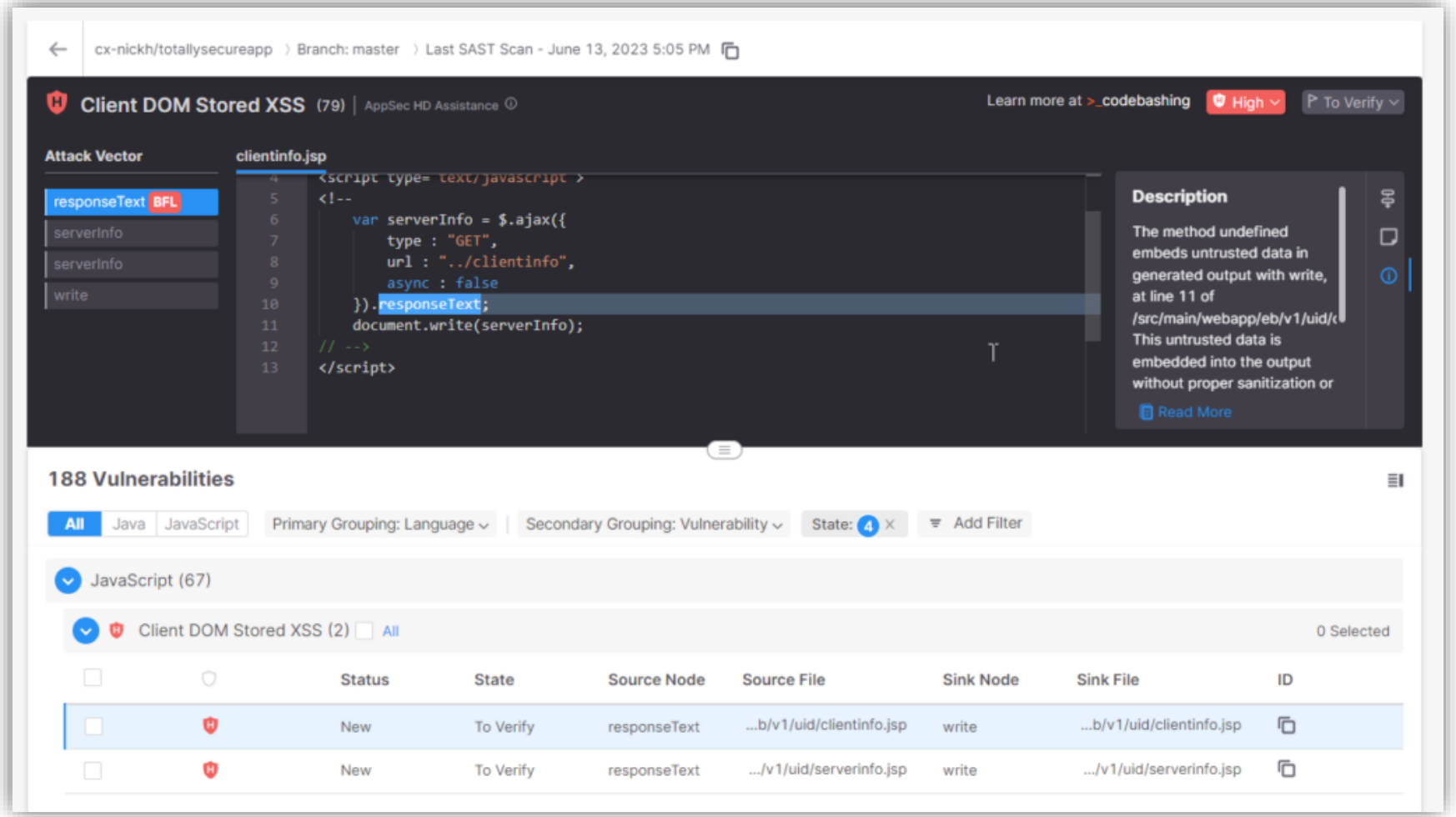
INTEGRAÇÃO SAST AO PIPELINE



Fonte: https://docs.checkmarx.com. Integração do Checkmarx One com o Azure DevOps.

PUC Minas

INTEGRAÇÃO SAST AO PIPELINE



Fonte: https://docs.checkmarx.com. Identificação de uma vulnerabilidade de Stored XSS. O BFL (Best Fix Location) indica o **melhor lugar para resolver o problema**.

ASPCTOS-CHAVE DA ESCOLHA DA SOLUÇÃO DE SAST

- Suporte às linguagens de programação adotadas pela organização
- Alta acurácia e baixa taxa de falso-positivo
- **Tempo de scan**
- Otimização dos filtros e regras de scan (código de teste, Objeto Mock e Stub, arquivos de imagem, documentos, entre outros)
- Avaliação de SCA, Contêineres e Infraestrutura como Código (IaC)
- Interface amigável e fácil gestão
- Definição de Papéis e Responsabilidades

ASPCTOS-CHAVE DA ESCOLHA DA SOLUÇÃO DE SAST

- Suporte a integração com as soluções de Pipeline em Cloud (Azure, AWS, GitHub) e On-premises (Jenkins) existentes
- Suporte para integração de ferramentas de colaboração e ITSM (Jira, ServiceNow, Azure Boards) existentes
- Suporte a consumo de API para consumo de informações, coleta de métricas e geração de indicadores
- Integração com Plugin de IDE existentes das estações de trabalho e **treinamento**
- Relatórios e Analytics (principalmente, risco dos projetos)

ACURÁCIA

EVENTO	SIGNIFICADO
Falso Positivo	Identificado como um problema, quando na verdade não é. **Baixa Acurácia.**
Falso Negativo	Problemas potenciais que não são identificados. **Pior caso.**
Verdadeiro Positivo	O problema identificado é real. **Melhor caso.**
Verdadeiro Negativo	Geralmente informativo. Comportamento normal e esperado.

Muitos **falsos positivos** perpetuam a fadiga e podem tornar o trabalho de análise de segurança repetitivo e estressante, impactando potencialmente a eficácia das atividades. **Falso negativo** pode fazer com que vulnerabilidades críticas na Aplicação não sejam identificadas, sendo causa de eventual exploração.

ACURÁCIA

$$j = \frac{\text{true positives}}{\text{true positives + false negatives}} + \frac{\text{true negatives}}{\text{true negatives + false positives}}$$

O OWASP Benchmark usa o **Índice de Youden** para medir a precisão da ferramenta. Esta fórmula irá gerar um número entre -1 e 1. O resultado mais próximo de 1 denota nenhum falso positivo, enquanto -1 denota apenas falso positivo.

PUC Minas

DESAFIO

DESAFIO

Uma empresa adotou uma solução de SAST que avalia vulnerabilidades no código-fonte por meio de fraquezas adotadas pela OWASP, CWE e outros. Nos primeiros seis meses de adoção da solução de SAST, a empresa resolveu adotar a quebra de Build. Uma das Squads apresentou forte descontentamento por não conseguir remediar as vulnerabilidades. **Quais as possíveis razões deste sintoma?**

DESAFIO

Uma empresa está realizando uma PoC de uma solução de SAST. Ao executar o primeiro scan no projeto de software, foram encontradas vulnerabilidades de SCA e IaC, contudo, mesmo analisando aproximadamente cinco mil linhas de código, não foram encontradas vulnerabilidades. **Quais as possíveis razões deste sintoma?**

PUC Minas

PUC Minas

REVISÃO MANUAL DE CÓDIGO

REVISÃO MANUAL DE SEGURANÇA DE CÓDIGO

- Processo de leitura do código-fonte linha por linha para identificar possíveis vulnerabilidades (pode-se adotar a estratégia de revisão em pares)
- Requer muita habilidade, experiência e paciência
- Problemas identificados nesta revisão ajudarão a organização a aumentar sua eficiência e esclarecer o contexto das decisões de codificação
- Foco em vulnerabilidades como problemas de lógica de negócios, autenticação, autorização, criptografia e validação geral de dados. Deve-se inspecionar caminhos de código raramente percorridos

REVISÃO MANUAL DE SEGURANÇA DE CÓDIGO

- Pode encontrar código que atua como backdoor ou que realiza ações de fraude
- Pode encontrar vulnerabilidades que SAST, DAST e Pentest eventualmente não encontrariam
- Pontos de atenção que devem ser revisados:
  - Processo de validação de dados
  - Tratamento de erros
  - Logging
  - Autenticação, Autorização e Gerenciamento de Sessão
  - Criptografia

REVISÃO MANUAL DE SEGURANÇA DE CÓDIGO

Qual fluxo de autorização é menos problemático?

```
if admin == false
(**limited access**)
else (**privileged access**)
```

A

```
if admin == true
(**privileged access**)
else (**limited access**)
```

B

SOURCES E SINKS

- **Sources** são quaisquer componentes, funções ou variáveis que permitem a entrada de dados externos no ecossistema da aplicação
  - Parâmetros de requisições HTTP (como dados enviados via GET, POST, cabeçalhos ou cookies)
  - **Campos de formulários** Web preenchidos por usuários
  - Dados lidos de arquivos externos, bancos de dados compartilhados ou APIs de terceiros

SOURCES E SINKS

- **Sinks** são as funções, métodos ou instruções internas que recebem dados (**sources**) e realizam alguma ação crítica com eles
- Se um dado malicioso atingir um sink sem tratamento, ocorre uma vulnerabilidade de segurança
  - Executores de consultas ao banco de dados podem gerar **SQL Injection**
  - Funções que executam comandos no sistema operacional (exec(), system()) podem gerar **Command Injection**
  - Renderizadores de interface ou HTML (innerHTML no JavaScript) podem gerar **Cross-Site Scripting (XSS)**

SOURCES E SINKS

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <title>Página de Busca</title>
</head>
<body>
  <h1>Resultados da busca para: <span id="termo-busca"></span></h1>

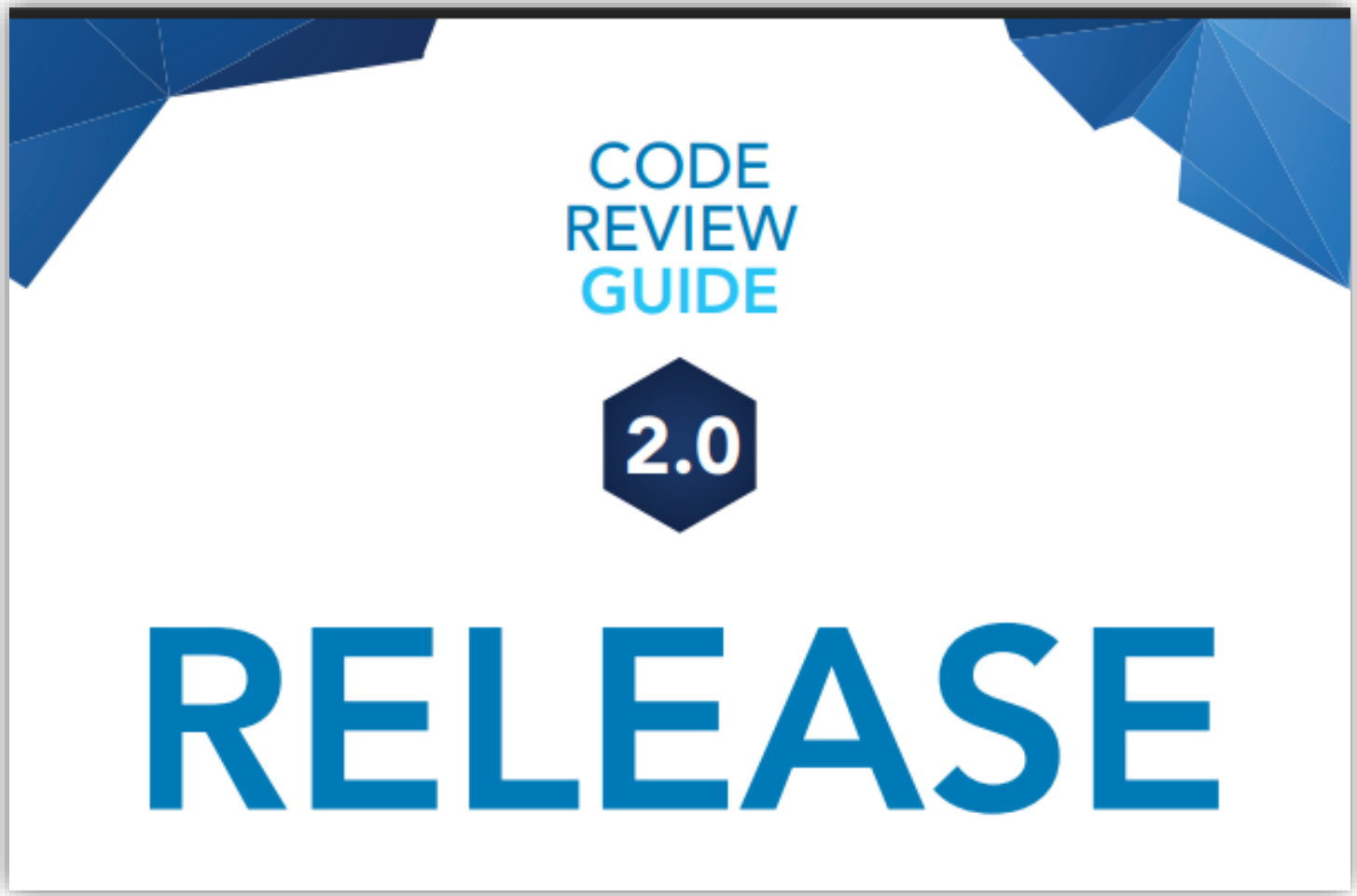
  <script>
    // SOURCE (Origem)
    // O script captura o que foi digitado na barra de endereços após '?q='
    // Esse dado vem direto do ambiente externo e não é confiável.
    const urlParams = new URLSearchParams(window.location.search);
    const busca = urlParams.get('q');

    // SINK (Destino)
    // A propriedade 'innerHTML' interpreta e renderiza qualquer string como código HTML/JS.
    // Como o dado da SOURCE é inserido diretamente aqui, ocoore a vulnerabilidade de XSS.
    document.getElementById("termo-busca").innerHTML = busca;
  </script>
</body>
</html>
```

REVISÃO MANUAL DE SEGURANÇA DE CÓDIGO

- Revisão manual de código deve ser parte da **cultura da empresa**
- Automatizações podem ser consideradas
- A revisão manual de código deve **preceder a aprovação do código-fonte**
- Deve ser um **requisito por meio de política específica por tema** (ou normas internas)
- Deve haver **treinamentos** sobre o tema de revisão de manual de código
- Geralmente é um **requisito para PCI-DSS, HIPAA**, outros

REVISÃO MANUAL DE SEGURANÇA DE CÓDIGO



Fonte: https://owasp.org/www-project-code-review-guide/

PUC Minas

DESAFIO

DESAFIO

Uma empresa está receosa de que a quebra de build, devido a baixa maturidade para lidar com as remediações de vulnerabilidades tenha um impacto indesejado. Como o processo de revisão de código e outros mecanismos podem ser utilizados enquanto a maturidade esperada é trabalhada?

PUC Minas

PUC Minas

DYNAMIC APPLICATION SECURITY TESTING

DYNAMIC APPLICATION SECURITY TESTING (DAST)

- Ferramentas DAST são adotadas para avaliar aplicações no **estado de execução**
- Foco em vulnerabilidades OWASP TOP 10, CWE e CVE
- Pode gerar prova de exploração
- Encontra vulnerabilidades críticas como XSS, SQLi, Command Injection, Code Injection
- **IMPORTANTE**: Ferramentas DAST podem causar interrupção em serviços, por isso, deve-se adotar na fase de Teste. Evitar o uso de DAST em ambiente de Produção tanto na perspectiva de segurança (shift-left) quanto da disponibilidade

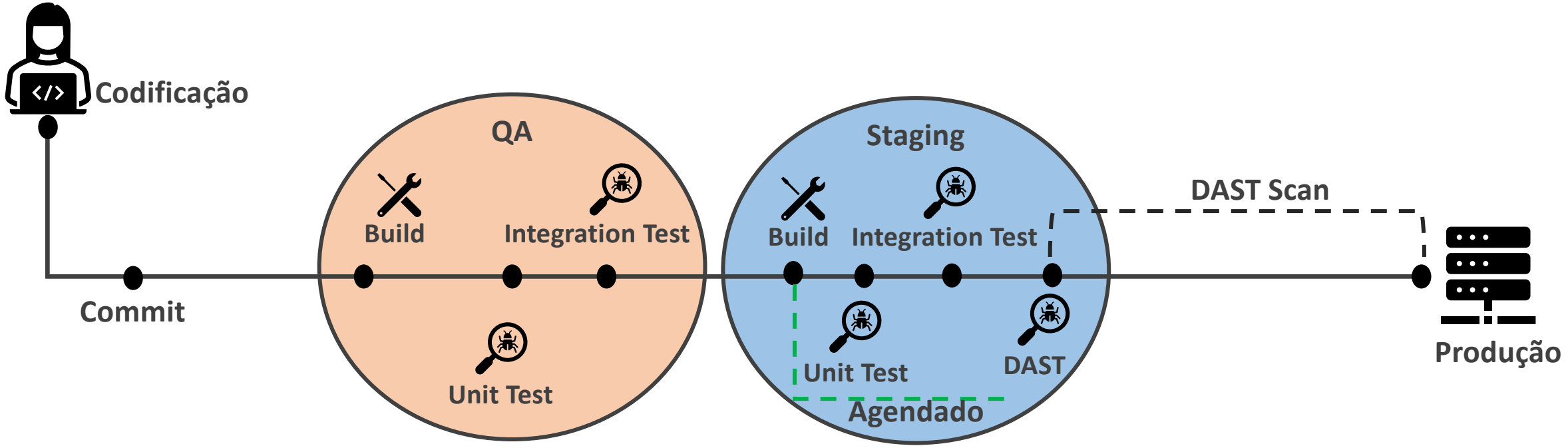
DYNAMIC APPLICATION SECURITY TESTING (DAST)

- Ferramentas **SAST** são ditas **White-Box** e ferramentas **DAST** são ditas **Black-Box**
- Deve ser integrado ao Pipeline DevOps principalmente para a fase de Test
- A solução DAST deve ser devidamente configurada para refletir sobre o teste, tais como a linguagem da programação, tecnologias, tipos de avaliações esperadas, entre outros
- A solução de DAST deve suportar **scan autenticado** e suporte a scan de API

DYNAMIC APPLICATION SECURITY TESTING (DAST)

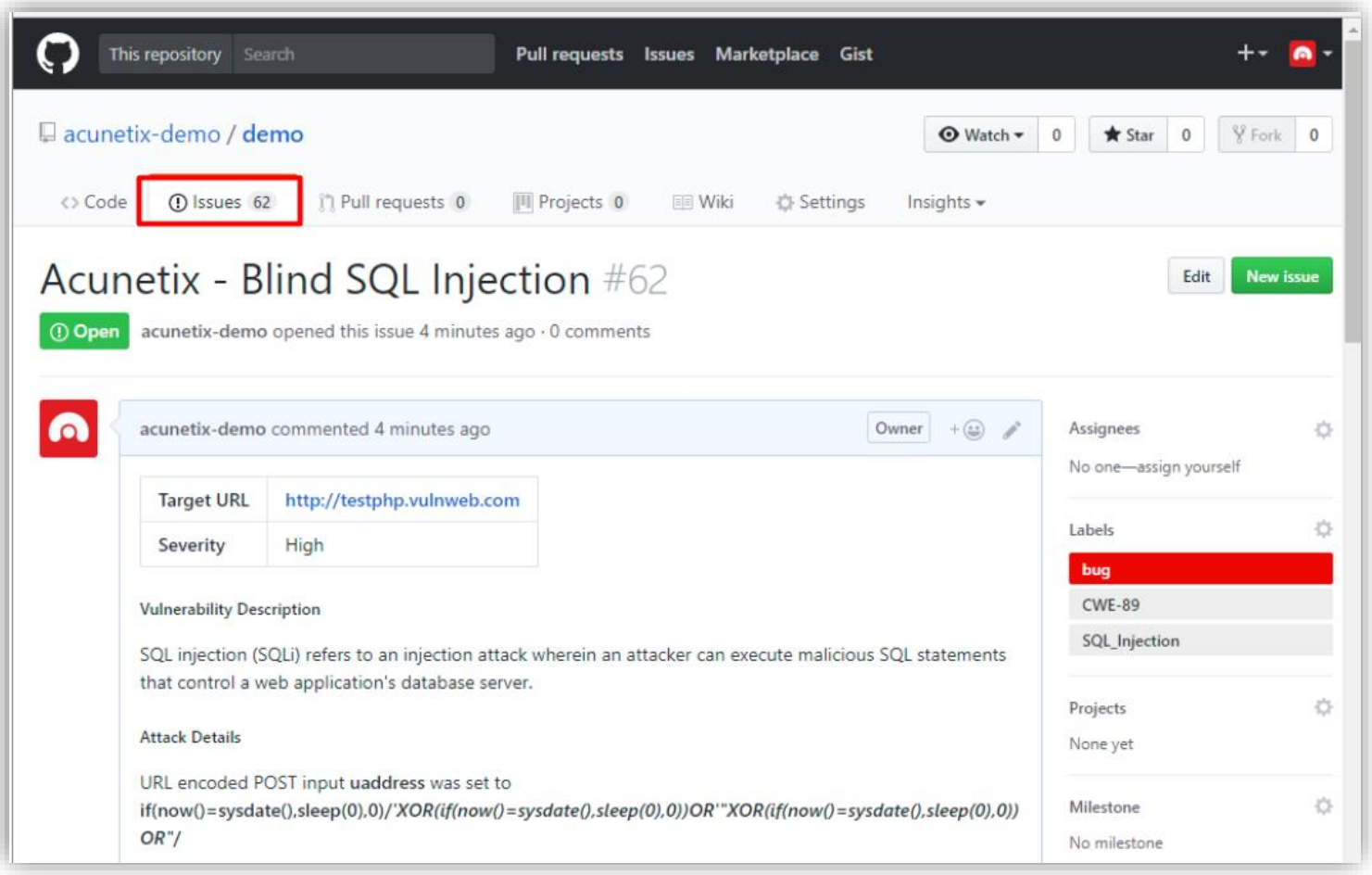
- A solução de DAST deve suportar Single Page Application (SPA)
- Pode haver problemas com aplicações requerem MFA
- Pode haver problemas com aplicações com muitos formulários
- Pode haver problemas com resoluções de CAPTCHA
- Possui menos falso-positivo que soluções de SAST

DYNAMIC APPLICATION SECURITY TESTING (DAST)



- O DAST pode ocorrer sempre durante uma tarefa de implantação em Staging/QA
- O DAST pode ocorrer de forma **agendada** no Staging/QA

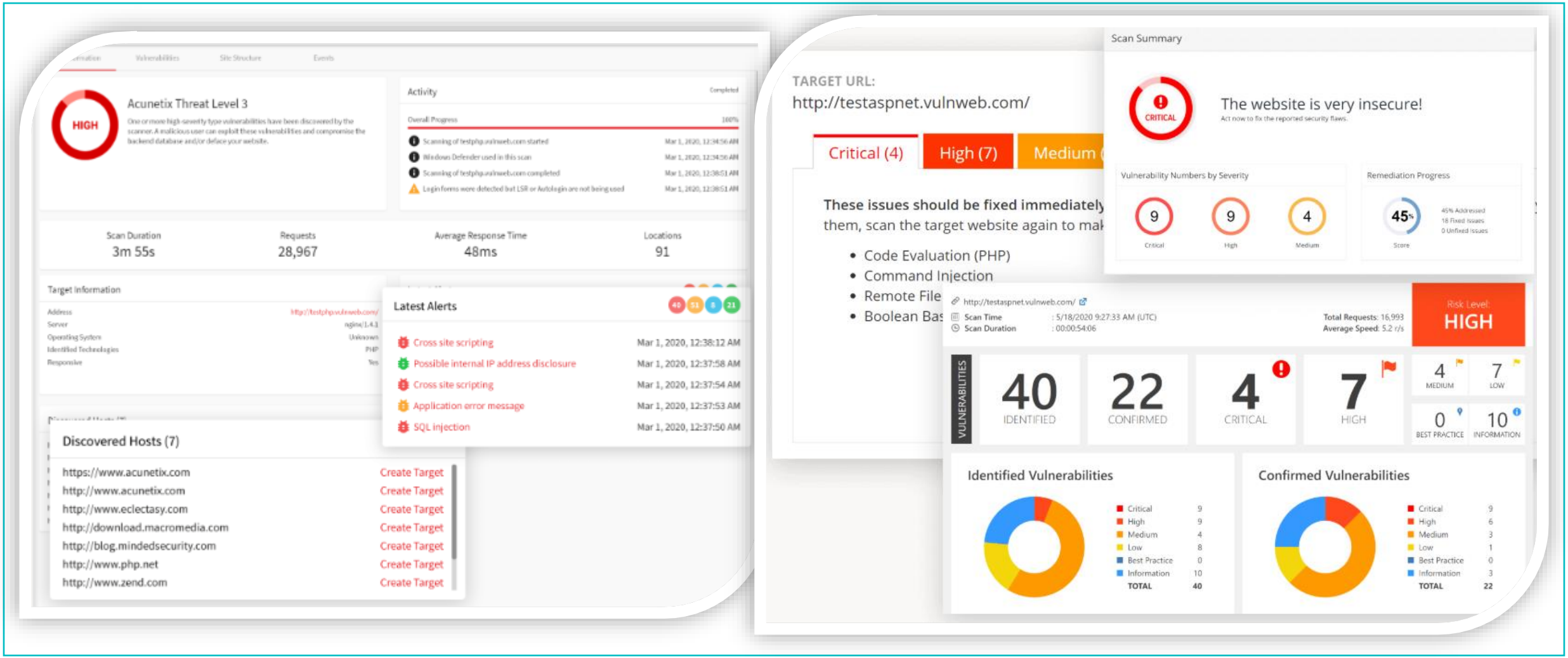
DYNAMIC APPLICATION SECURITY TESTING (DAST)



- O DAST pode ser integrado ao SCM. Ao lado, integração com o GitHub recebendo Issues para ser analisado

Fonte: https://www.acunetix.com/blog/articles/dast-dynamic-application-security-testing/

DYNAMIC APPLICATION SECURITY TESTING (DAST)



Fonte: https://www.acunetix.com/product/acunetix360/

PUC Minas

DYNAMIC APPLICATION SECURITY TESTING (DAST)











PUC Minas

DESAFIO

DESAFIO

Uma empresa está adotando uma solução de DAST integrada ao Pipeline. A política estabelecida é que se vulnerabilidades com severidade Crítica forem encontradas, o Pipeline deve ser quebrado. Contudo, inicialmente, o tempo de scan do DAST na tarefa está demorando entre 45 minutos e 120 minutos, dependendo do projeto. **O que pode ser feito nestas condições para reduzir o tempo de scan e medidas para minimizar o impacto garantindo a segurança do projeto de software?**

DESAFIO

Uma empresa está adotando uma solução de DAST não integrada ao Pipeline e executa o scan no ambiente de Produção. Um primeiro scan foi agendado para ocorrer às 2h da madrugada. Quando as operações começaram na empresa, a equipe tomou contato com o relatório que apresentou diversas vulnerabilidades encontradas, contudo, a aplicação teve modificações inesperadas e comportamentos anômalos. **Quais as possíveis razões do ocorrido?**

PUC Minas

PUC Minas

INTERACTIVE APPLICATION SECURITY TESTING

INTERACTIVE APPLICATION SECURITY TESTING (IAST)

- IAST adota a Instrumentação de Software como forma de operação
- Uma porção de código é adicionada ao aplicativo para coletar dados e realizar análises, integrando lógica de coleta ao código
- Soluções de APM (Application Performance Monitoring) adotam instrumentação de código, tais como Newrelic, Datadog, Dynatrace, Splunk
- A instrumentação de software pode causar aumento de CPU e lentidão
- **IMPORTANTE**: IAST só deve ser usado em **ambiente não produtivo**

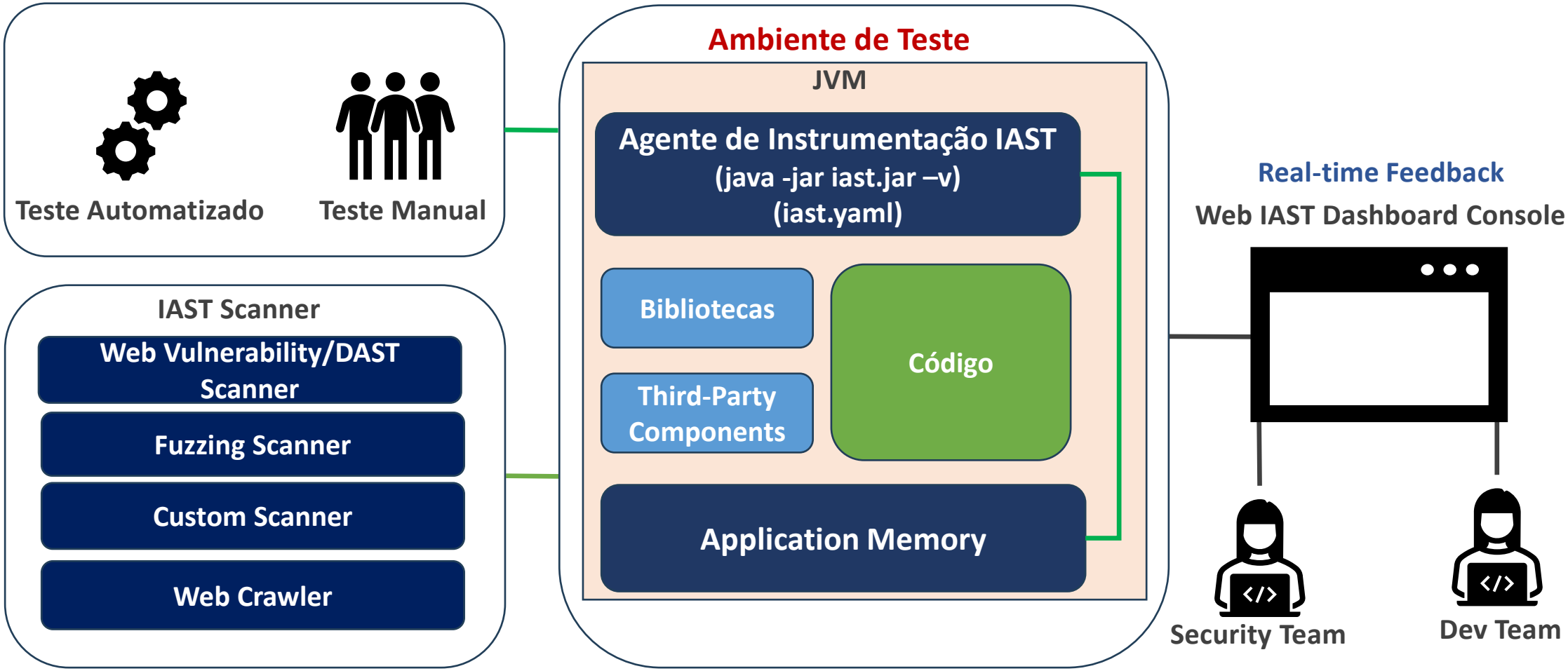
INTERACTIVE APPLICATION SECURITY TESTING (IAST)

- Tecnologia de teste de segurança em **tempo de execução** que identifica e diagnostica vulnerabilidades, monitorando continuamente em busca de vulnerabilidades (IAST adota um método **Gray-Box**)
- O IAST pode analisar o código em busca de vulnerabilidades em um aplicativo enquanto ele é executado por um teste automatizado ou outra atividade que possa acessar a funcionalidade do aplicativo
- Pode encontrar **vulnerabilidades conhecidas** e **não conhecidas**
- Geralmente, não possui falso-positivo

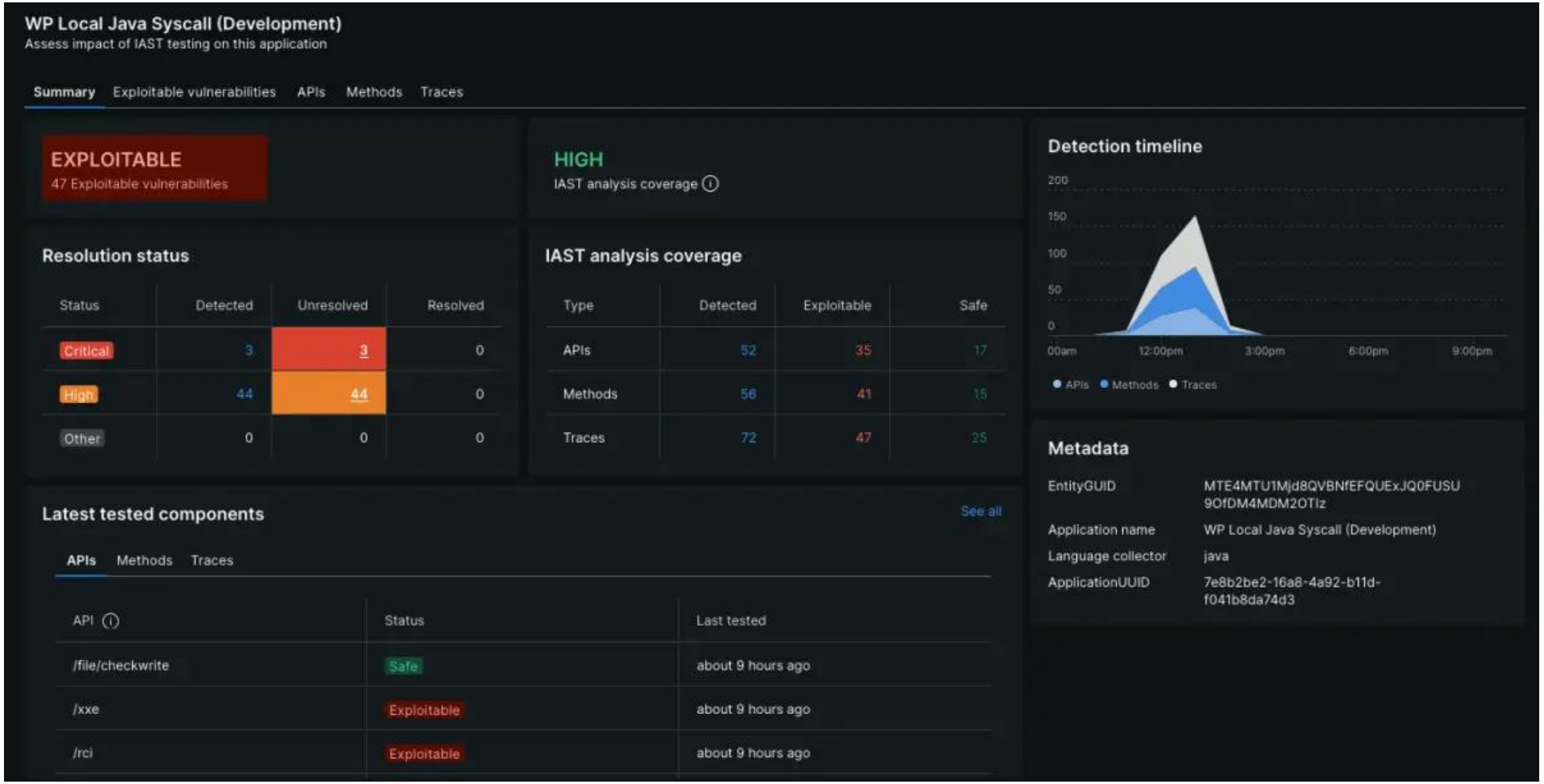
INTERACTIVE APPLICATION SECURITY TESTING (IAST)

- IAST analisa o código em busca de vulnerabilidades de segurança e as relata em tempo real
- Atua **dentro do aplicativo (instrumentação de software)**, o que o torna diferente tanto da análise estática (SAST) quanto da análise dinâmica (DAST). O IAST **testa apenas o que for especificado pelo teste funcional**, em vez de testar todo o aplicativo ou código
- O IAST tem acesso ao código e as requisições HTTP; portanto, ele pode executar SAST em cada linha de código e pode executar DAST em cada solicitação e resposta. IAST é um superconjunto de ferramentas SAST e DAST

INTERACTIVE APPLICATION SECURITY TESTING (IAST)



INTERACTIVE APPLICATION SECURITY TESTING (IAST)

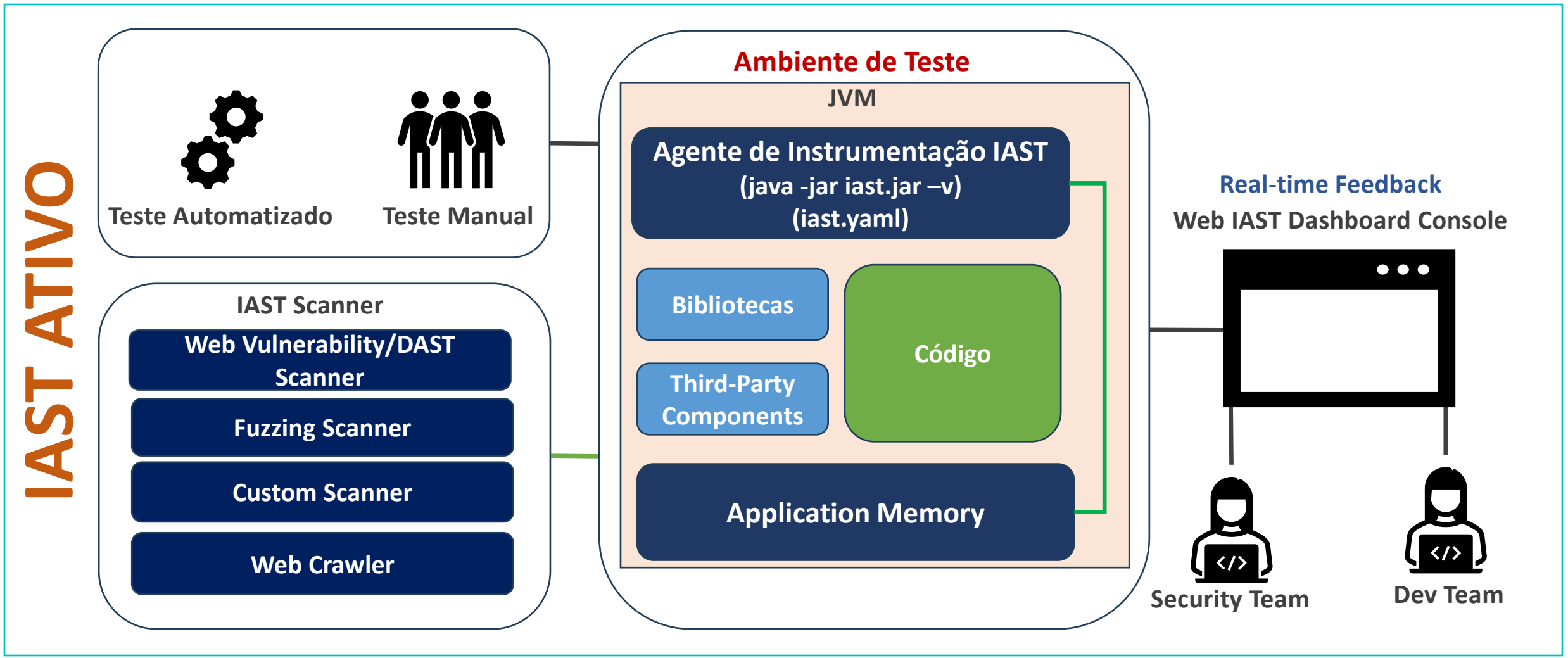


Fonte: https://newrelic.com

INTERACTIVE APPLICATION SECURITY TESTING (IAST)

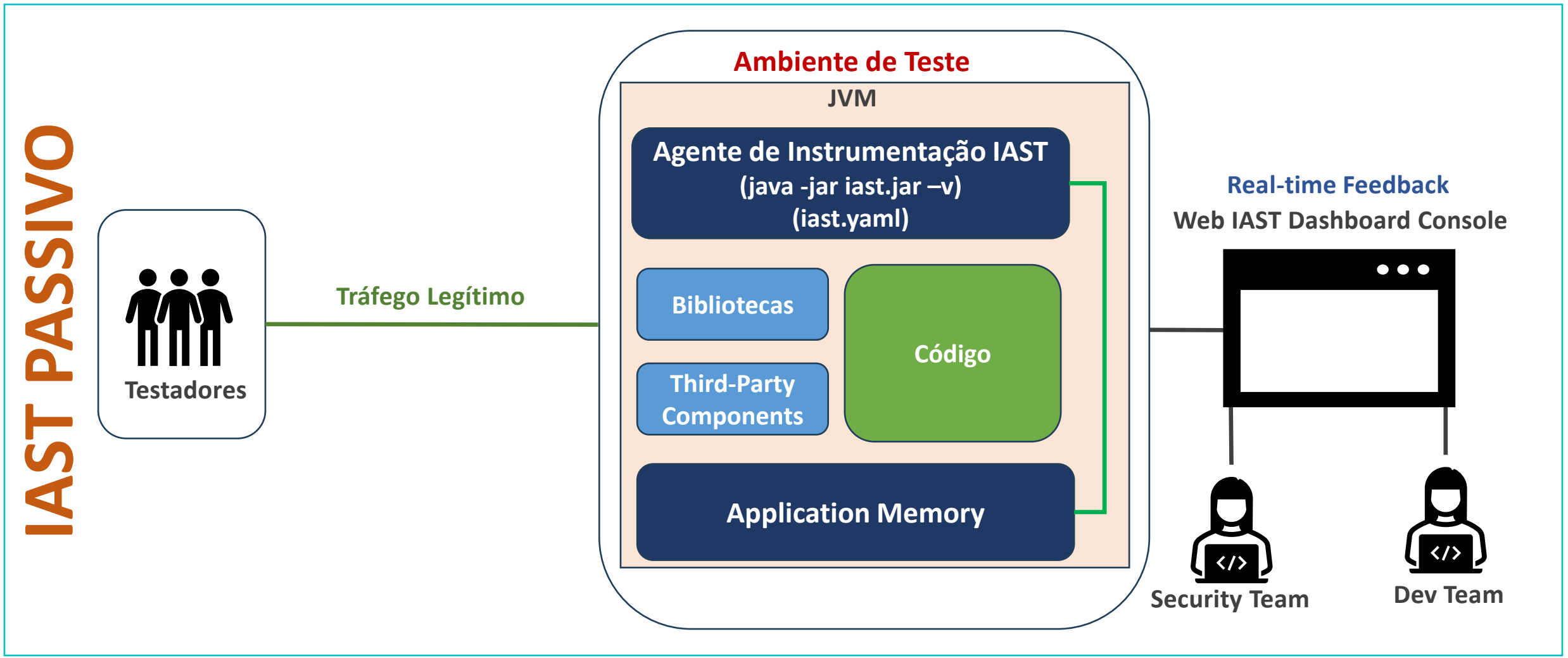
- **IAST Ativo (Partial IAST):** Combina um **scanner da Web com um agente** que funciona dentro do servidor de aplicativos que hospeda o aplicativo para fornecer detalhes de análise adicionais, como o local da vulnerabilidade
- **IAST Passivo (IAST Full):** Precisa **apenas** de **um componente de detecção** que é parte do servidor de aplicativos como um agente de tempo de execução

INTERACTIVE APPLICATION SECURITY TESTING (IAST)



PUC Minas

INTERACTIVE APPLICATION SECURITY TESTING (IAST)



PUC Minas

INTERACTIVE APPLICATION SECURITY TESTING (IAST)

PRINCIPAIS VULNERABILIDADES ENCONTRADAS COM IAST	
CATEGORIA	**TIPOS DE VULNERABILIDADES**
Injection	Incluem SQLi, Reflected XSS, Path Traversal, LDAP Injection, Log Injection, execução de código inseguro, Injeção de Entidade Externa XML (XXE), Header Injection, Injeção de NoSQL, Stored XSS, Command Injection, Xpath Injection, injeção de linguagem de expressão, injeção de hibernação.
HTTP Header	Incluem cache, resposta com cabeçalho de CSP de forma insegura, controles Anti-Clickjacking, HSTS, resposta sem cabeçalho de CSP, cookie inseguro, cabeçalho de resposta sem X-Content-Type, cabeçalho com versão de software ativado, resposta com cabeçalho de segurança de HSTS configurado de forma insegura e resposta com proteção X-XSS desativada, entre outros.
Configuração Insegura	Incluem verificação de cabeçalho desabilitada, mensagens de erro detalhadas, posicionamento JSP inseguro, validação de evento desabilitada, tempo limite de sessão excessivamente longo, modo de validação de solicitação desabilitado, comprimento máximo de solicitação grande, rastreamento habilitado, rastreamento habilitado para ASPX, decodificação XML insegura, metadados de serviço habilitados, validação de solicitação desabilitada, entre outros.

INTERACTIVE APPLICATION SECURITY TESTING (IAST)

PRINCIPAIS VULNERABILIDADES ENCONTRADAS COM IAST	
CATEGORIA	**TIPOS DE VULNERABILIDADES**
Parsing	Incluem entidades externas XML, DoS de expressão regular, redirecionamento não validado, vinculação automática não verificada, desserialização insegura, poluição de parâmetro, uso de readLine em fluxos não confiáveis e violação de limite de confiança.
Autenticação e Autorização	Incluem senha codificada, cookie de sessão sem a flag “HttpOnly” e “Secure”, protocolo de autenticação inseguro, regravação de sessão, cadeias de conexão desprotegidas, redirecionamento de aplicativos cruzados de autenticação de formulários, IDs de sessão expirados não regenerados, modo de proteção de autenticação de formulários, formulários sem prevenção de preenchimento automático. No contexto de autorização, incluem modo de proteção do gerenciador de função, CSRF, fraqueza de adulteração de verbo HTTP, SSRF, entre outros.
Criptografia	Incluem validação de MAC desativada, algoritmos de hash inseguros, oráculo de preenchimento, algoritmos de criptografia inseguros, geração de números aleatórios fracos.

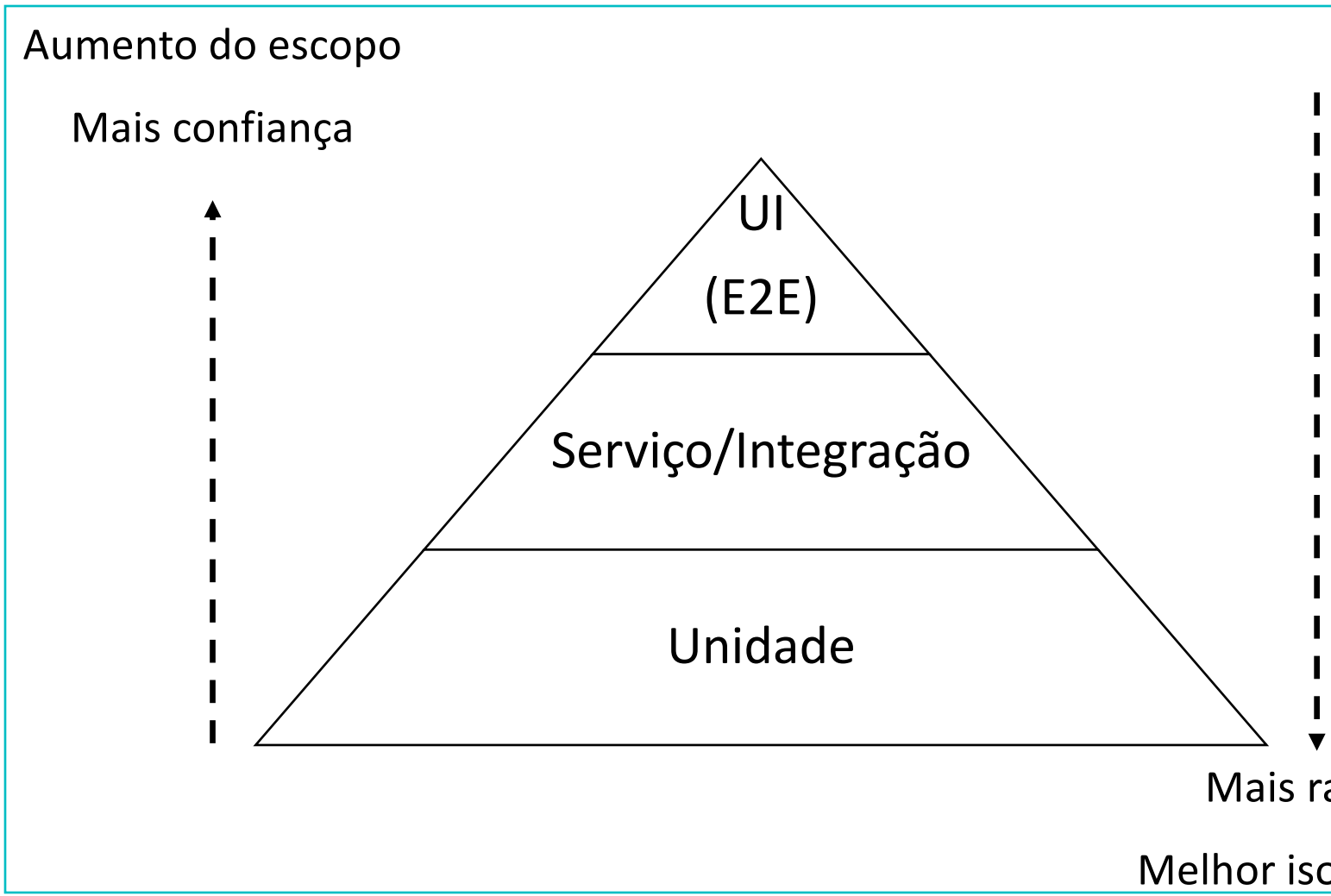
INTERACTIVE APPLICATION SECURITY TESTING (IAST)

- IAST deve ser integrado com ferramentas de ITSM ou de gestão de projetos de software, como JIRA, Azure Boards, ServiceNow etc. para abrir tickets e gerar backlog
- IAST deve ser ajustado para encontrar dados que lidam com aspectos legais e regulatórios como GDPR, PCI-DSS, LGPD
- IAST deve fornecer meios de correção adequados e fácil compreensão para os utilizadores
- IAST deve suportar as linguagens de programação e frameworks utilizados pela organização

TESTES DE SOFTWARE E SEGURANÇA

- Teste de Abuso
- Teste de Intrusão (Pentest)
- Teste Unitário
- Teste de Integração
- Teste de Regressão/Teste de Regressão de Segurança
- Smoke Test Automatizado de Segurança
  - Autenticação, Autorização, funções críticas do sistema

TESTES DE SOFTWARE E SEGURANÇA



- A adoção de microsserviços podem trazer novas perspectivas para lidar com os testes
- A base da pirâmide sempre deve constituir em escala os testes mais rápidos, caso contrário pode se tornar um antipadrão (*snow cone*, pirâmide invertida)

Fonte: COHN, Mike. Succeeding with Agile. Addison-Wesley, 2009. EUA.

TESTES DE SOFTWARE E SEGURANÇA

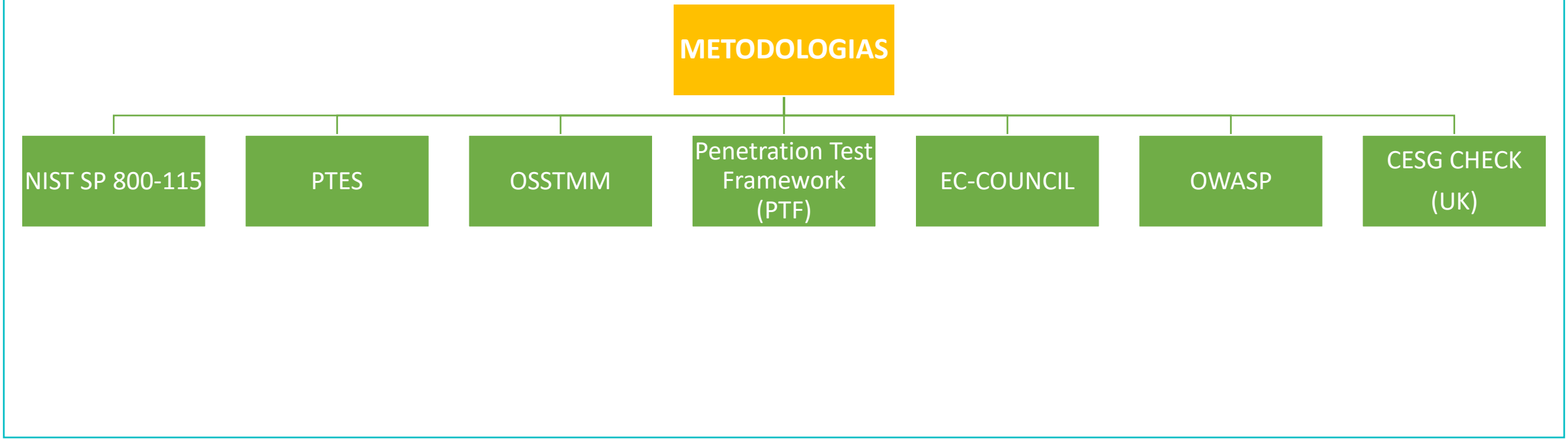
- Alguns autores (FOWLER, Susan J. Microsserviços Prontos Para a Produção. NOVATEC. 2017.) recomendam realizar os **testes de software antes da revisão de código** realizando todo processo de teste em uma Branch separada, fazendo com que o código que for aprovado já foi devidamente testado
- De certa forma, isso reduz o número de revisões em função de eventual refatoração de código em caso de falhas nos testes
- Nesse processo foi considerado o Teste Unitário, Teste de Integração e Teste Fim-a-Fim

PUC Minas

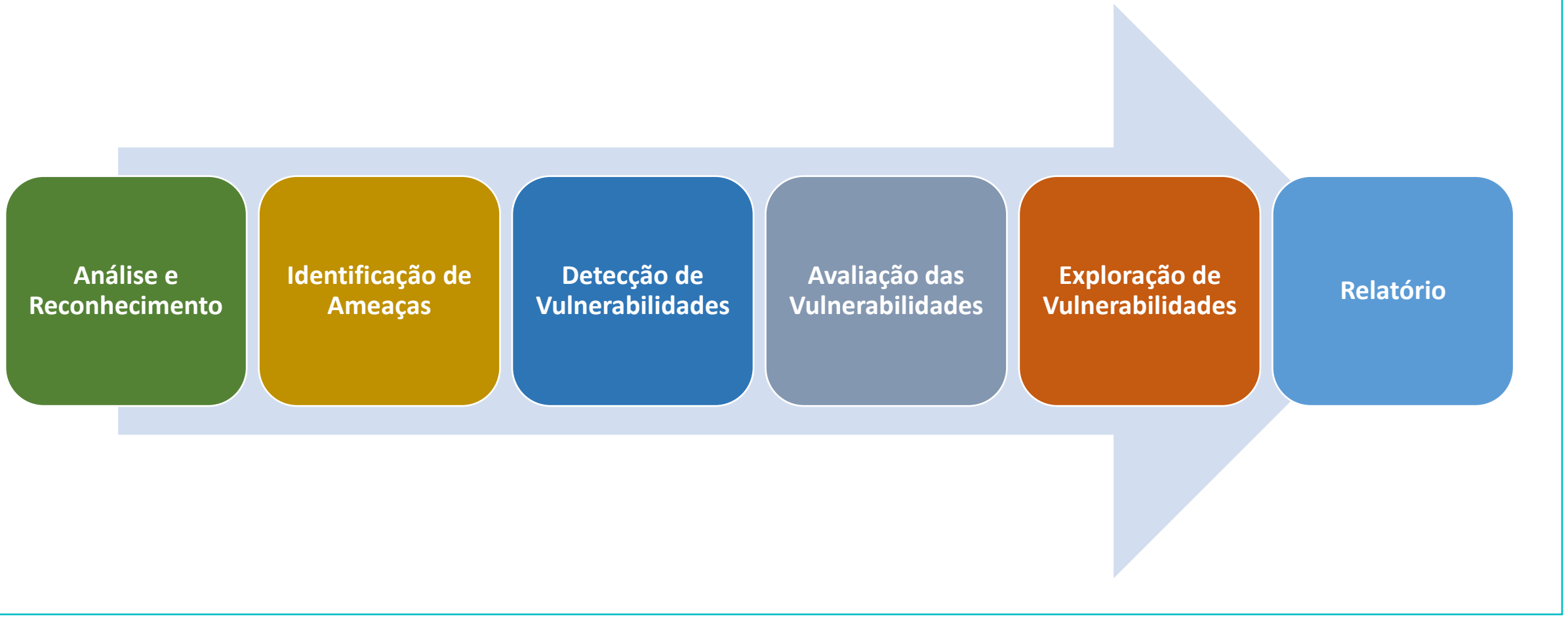
PUC Minas

TESTE DE INTRUSÃO

METODOLOGIAS DE PENTEST



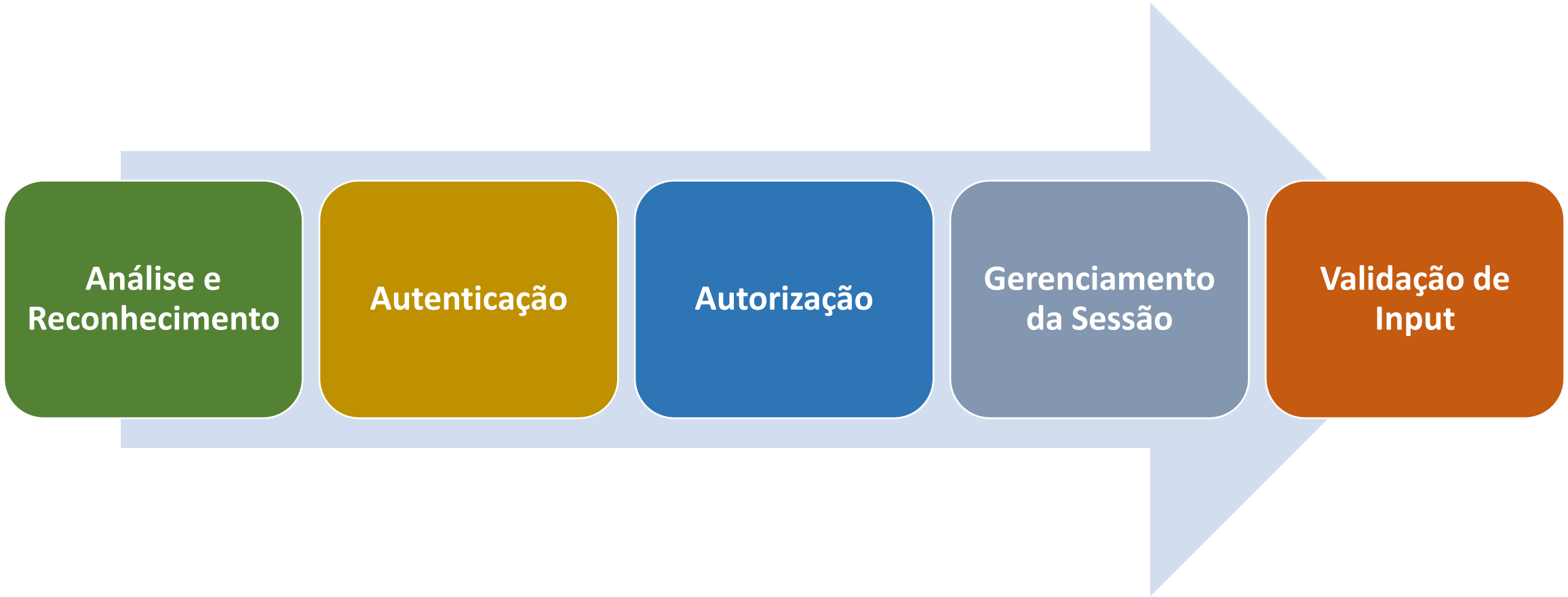
FASES TÍPICAS DO TESTE DE INTRUSÃO/PENTEST



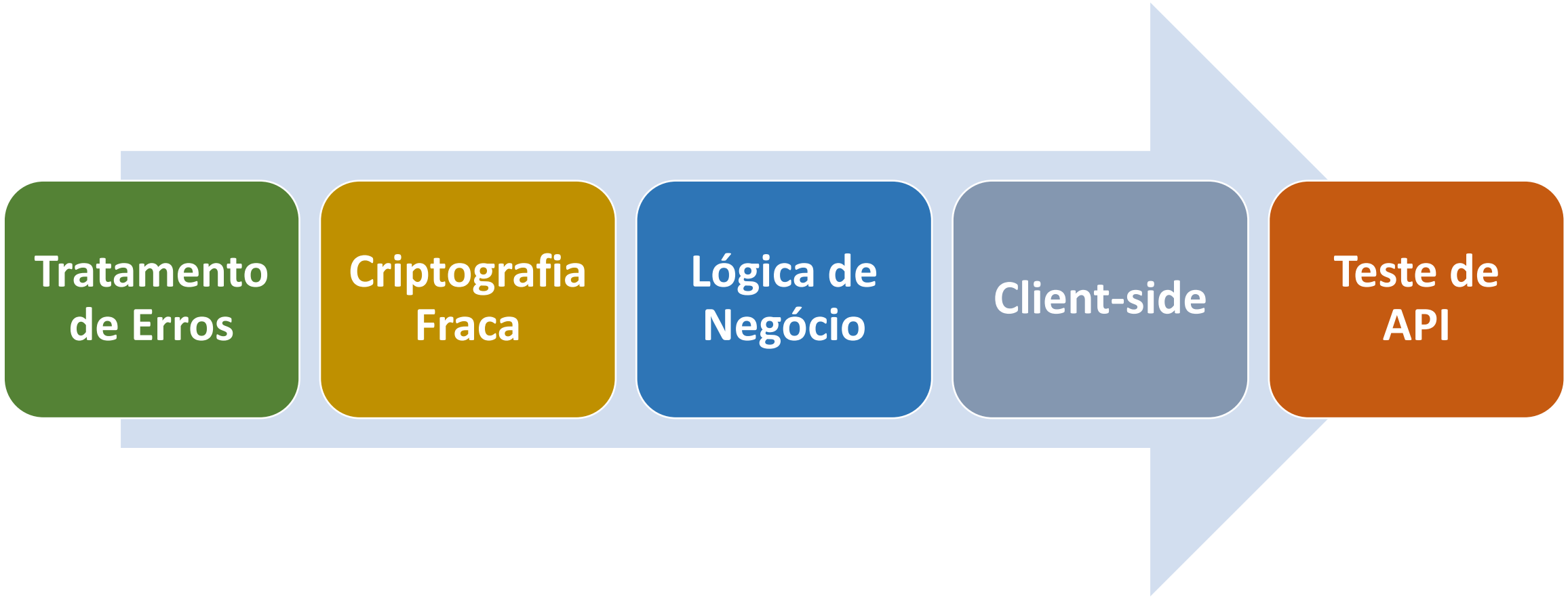
OWASP

- O OWASP (Open Web Application Security Project) é amplamente utilizado e possui três métodos para testes de segurança:
  - **OWASP Web Security Testing Guide**
  - OWASP Mobile Security Testing Guide
  - OWASP Firmware Security Testing Methodology

OWASP


Análise e Reconhecimento
Autenticação
Autorização
Gerenciamento da Sessão
Validação de Input

OWASP


Tratamento de Erros
Criptografia Fraca
Lógica de Negócio
Client-side
Teste de API

OWASP

Aplicação Vulnerável a Redirecionamento Inseguro

CVSS v3.0	Severidade
7.5	Alto

Descrição

Os redirecionamentos e encaminhamentos arbitrários são possíveis quando uma aplicação aceita uma entrada não confiável que pode fazer com que o sistema redirecione a solicitação para um endereço externo. Ao modificar a entrada de URL não confiável para um site malicioso, um invasor pode lançar com êxito um esquema de *phishing* e roubar as credenciais do usuário. Como o nome do servidor no *link* modificado é idêntico ao do site original, as tentativas de *phishing* podem ter uma aparência mais confiável. Ataques de redirecionamento e encaminhamento não validados também podem ser usados para criar de forma mal-intencionada uma URL que passaria na verificação de controle de acesso do aplicativo e, em seguida, encaminhará o invasor para funções privilegiadas que ele normalmente não conseguiria acessar.

Referências

- CWE
  - CWE-601: URL Redirection to Untrusted Site ('Open Redirect')

Mais Informações

- WSTG-CLNT-04 - Testing for Client Side URL Redirect

PUC Minas

PROGRAMAS DE BUG BOUNTY



HACKA/FLAG

bugcrowd
Crowdsourced Security

hackerone

PUC Minas

PUC Minas

SECURITY SDLC: RELEASE E DEPLOY

RELEASE E DEPLOY

- Esta fase adota tecnologias de RASP
- Caracterizada pela forte adoção de Infraestrutura como Código (IaC)
- Gestão do ambiente das aplicações por gestão de configuração
- Segurança de gerenciadores de contêineres, dos contêineres e imagens de contêineres
- Adoção de Pentest

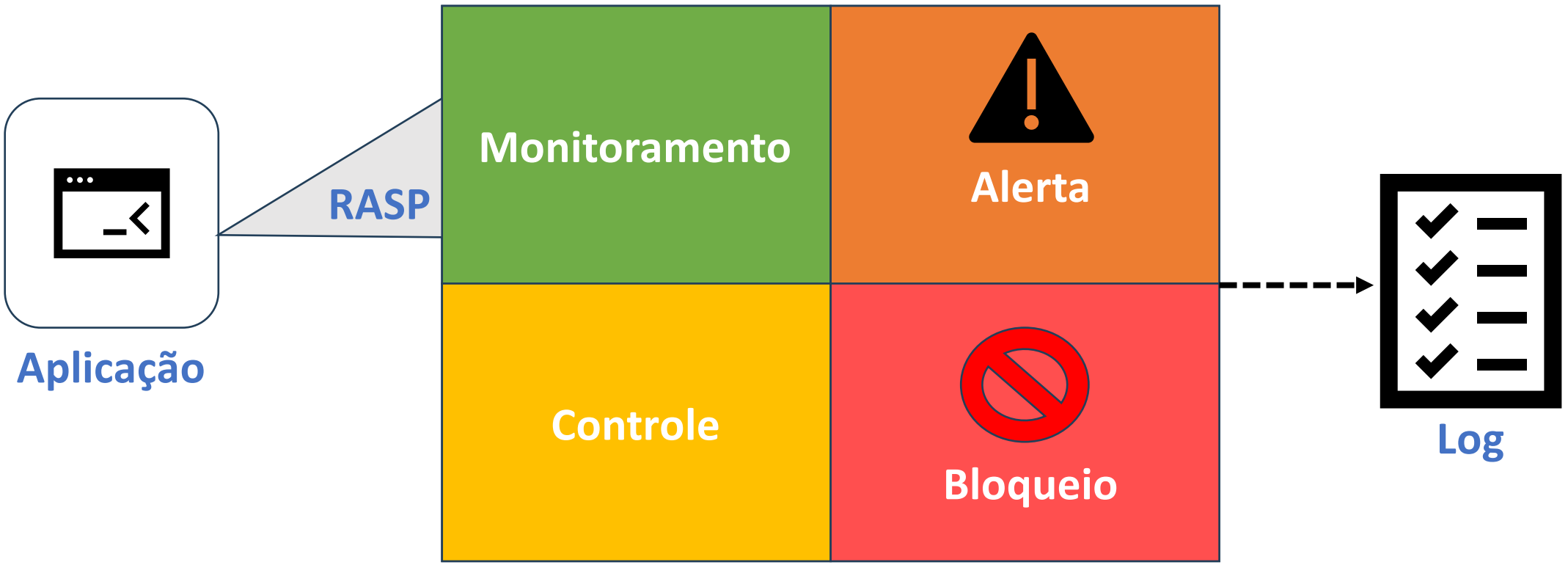
PUC Minas

RUNTIME APPLICATION SELF-PROTECTION (RASP)

RUNTIME APPLICATION SELF-PROTECTION (RASP)

- Tecnologia de segurança construída dentro de um aplicativo ou ambiente de tempo de execução do aplicativo usado em ambiente de **Homologação** e **Produção**
- Pode controlar a execução de um aplicativo para detectar e prevenir ataques em tempo real com baixíssimo (ou inexistente) falso-positivo
- Protege o aplicativo contra explorações implementado segurança em tempo de execução do aplicativo analisando o comportamento do aplicativo e o contexto do comportamento
- Amplamente adotado para proteção de aplicações móveis críticas

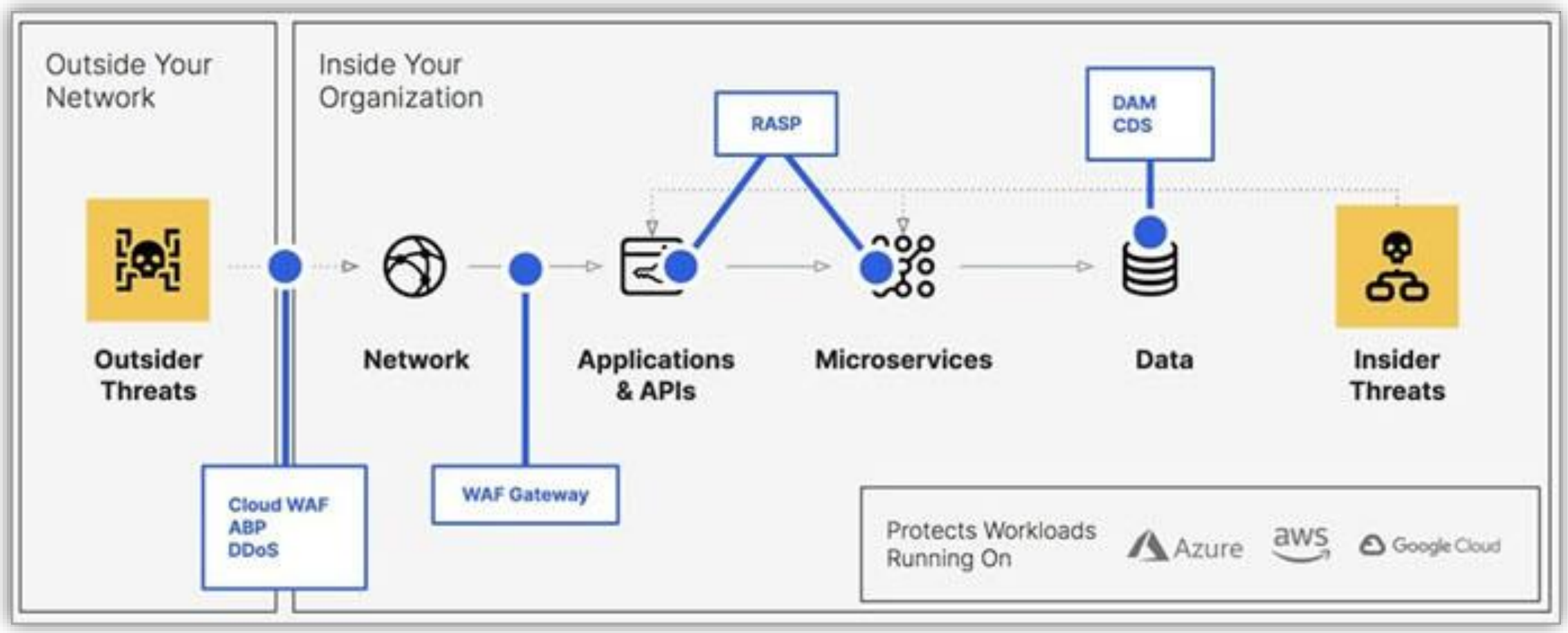
RUNTIME APPLICATION SELF-PROTECTION (RASP)



RUNTIME APPLICATION SELF-PROTECTION (RASP)

- Pode funcionar nos seguintes modos:
  - Desligado
  - Monitoramento
  - Bloqueio (comportamento)
  - Bloqueio Perimetral (regras pré-determinadas)
- O RASP deve ser integrado no Pipeline no ciclo de desenvolvimento de sistemas

RUNTIME APPLICATION SELF-PROTECTION (RASP)

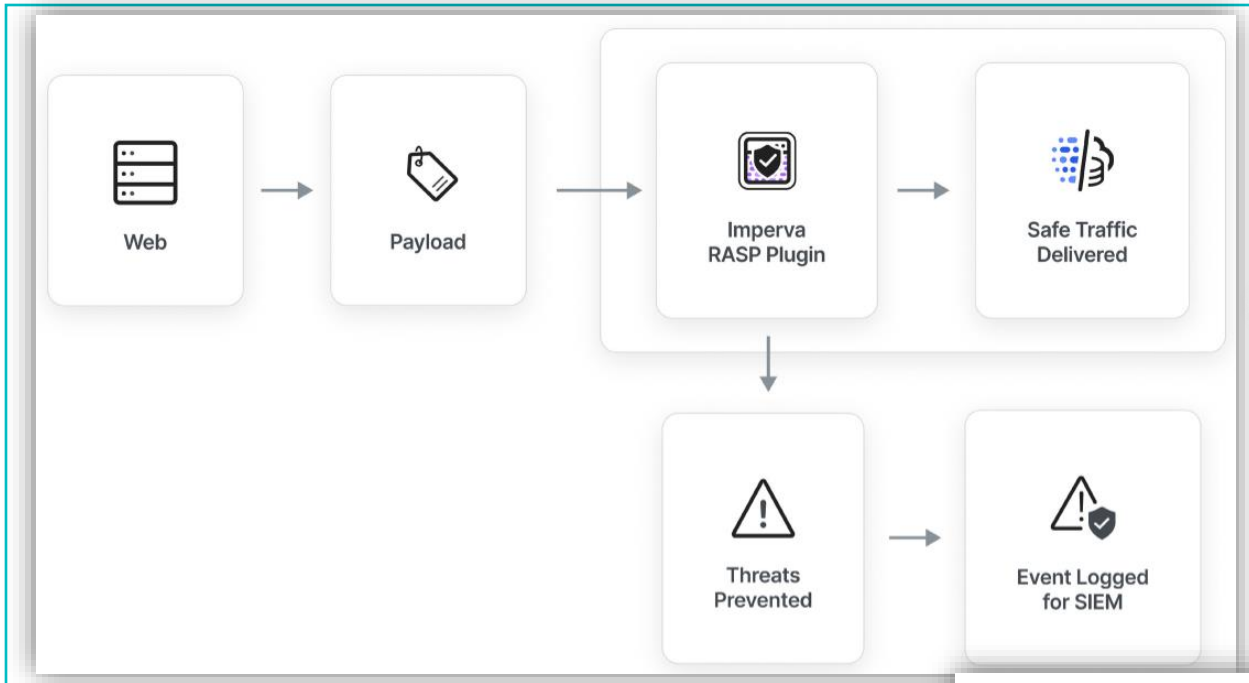


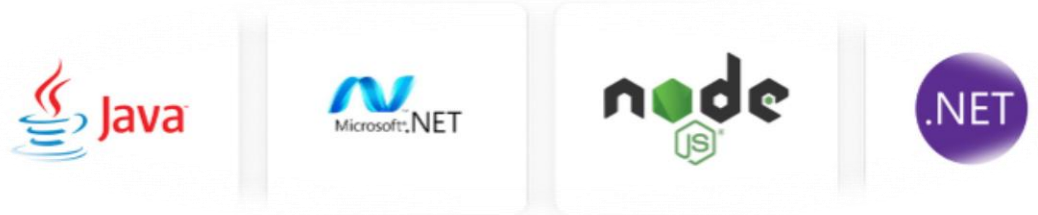
Fonte: https://www.softwaretestinghelp.com

RUNTIME APPLICATION SELF-PROTECTION (RASP)

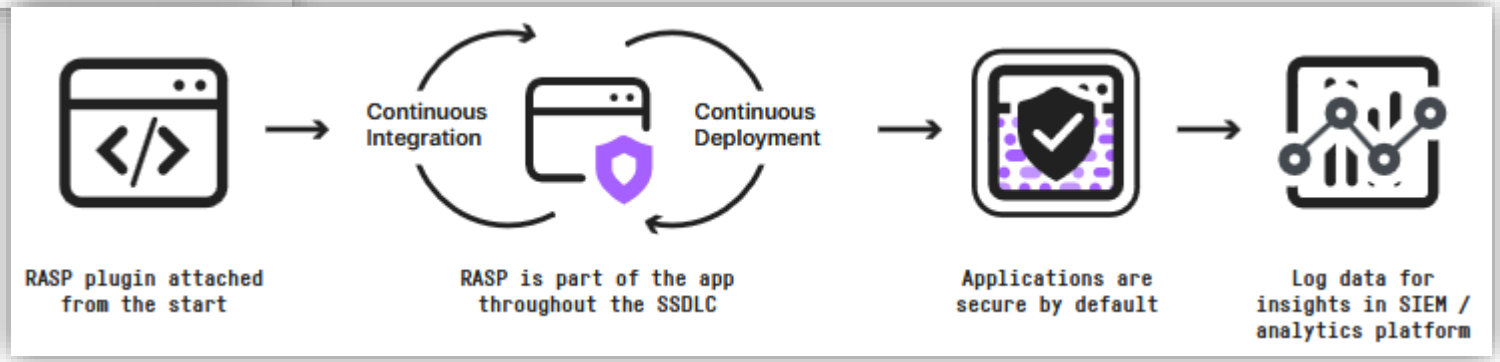
- Abordagens de segurança RASP
  - **Servlet Filters, SDK e Plugins:** filtros ou plugins são adicionados para analisar as solicitações HTTP e inspeção do payload para bloqueio
  - **Instrumentação binária**: assim como no IAST, para monitorar a aplicação
  - **Substituição de JVM:** pode substituir bibliotecas padrão como JAR ou JVM por uma camada RASP
  - **Virtualização**: uma cópia da aplicação é gerada e regras são estabelecidas e replicadas na aplicação original

RUNTIME APPLICATION SELF-PROTECTION (RASP)





O RASP permite que as organizações implantem software por meio do conceito de **Secure by Default**



Fonte: https://imperva.com

RUNTIME APPLICATION SELF-PROTECTION (RASP)

- Command injection
- Clickjacking
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF/XSRF)
- Database Access Violation (Advanced SQLI)
- HTML Injection
- HTTP Method Tampering
- HTTP Response Splitting
- Insecure Cookies
- Insecure Transport
- JSON Injection
- Large Requests
- Logging Sensitive Information
- Malformed Content-Types
- OGNL Injection
- Path Traversal
- SQL Injection
- Logging Sensitive Info
- Insecure Transport Protocol
- Unauthorized Network Activity
- Uncaught Exceptions
- Unvalidated Requests
- Vulnerable Dependencies
- Weak Authentication
- Weak Browser Cache Management
- Weak Cryptography & Ciphers
- XML External Entity Injection (XXE)
- XML Injection

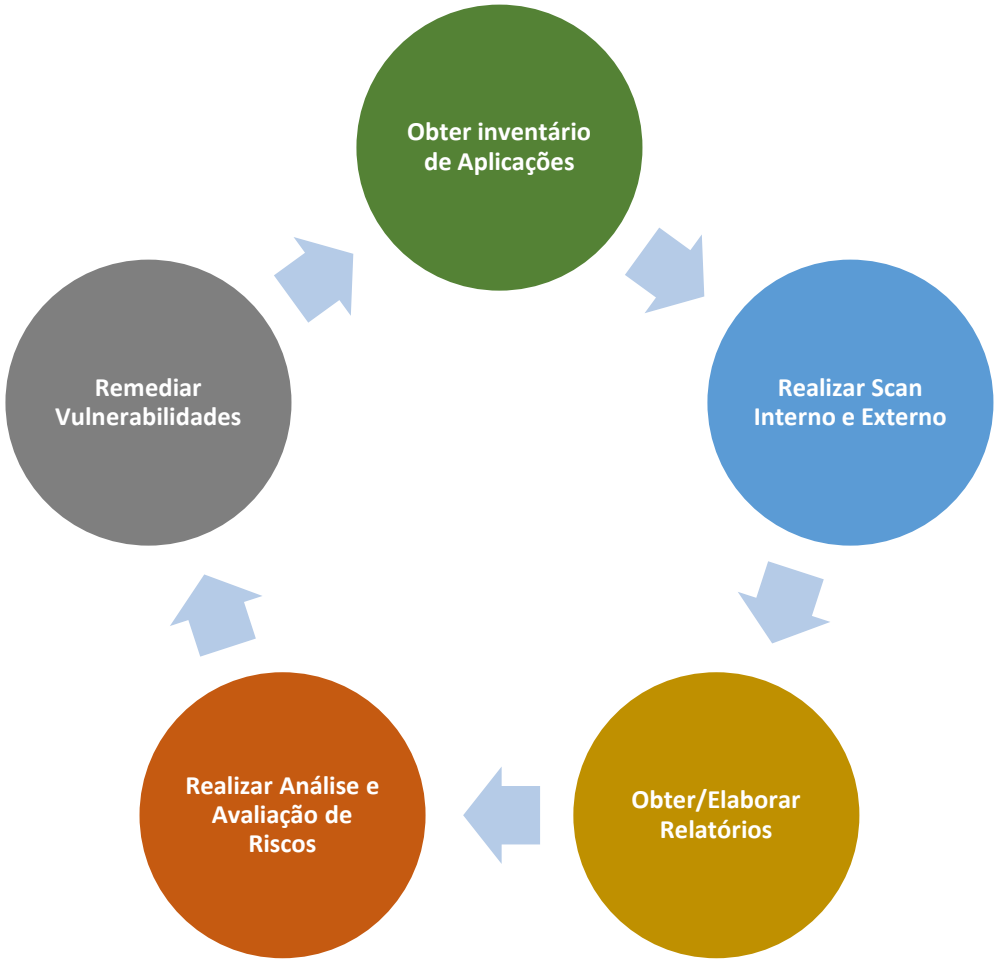
Fonte: https://imperva.com. **Principais vulnerabilidades e ameaças protegidas por soluções de RASP**.

PUC Minas

PUC Minas

ANÁLISE CONTÍNUA DE VULNERABILIDADES

PROCESSO

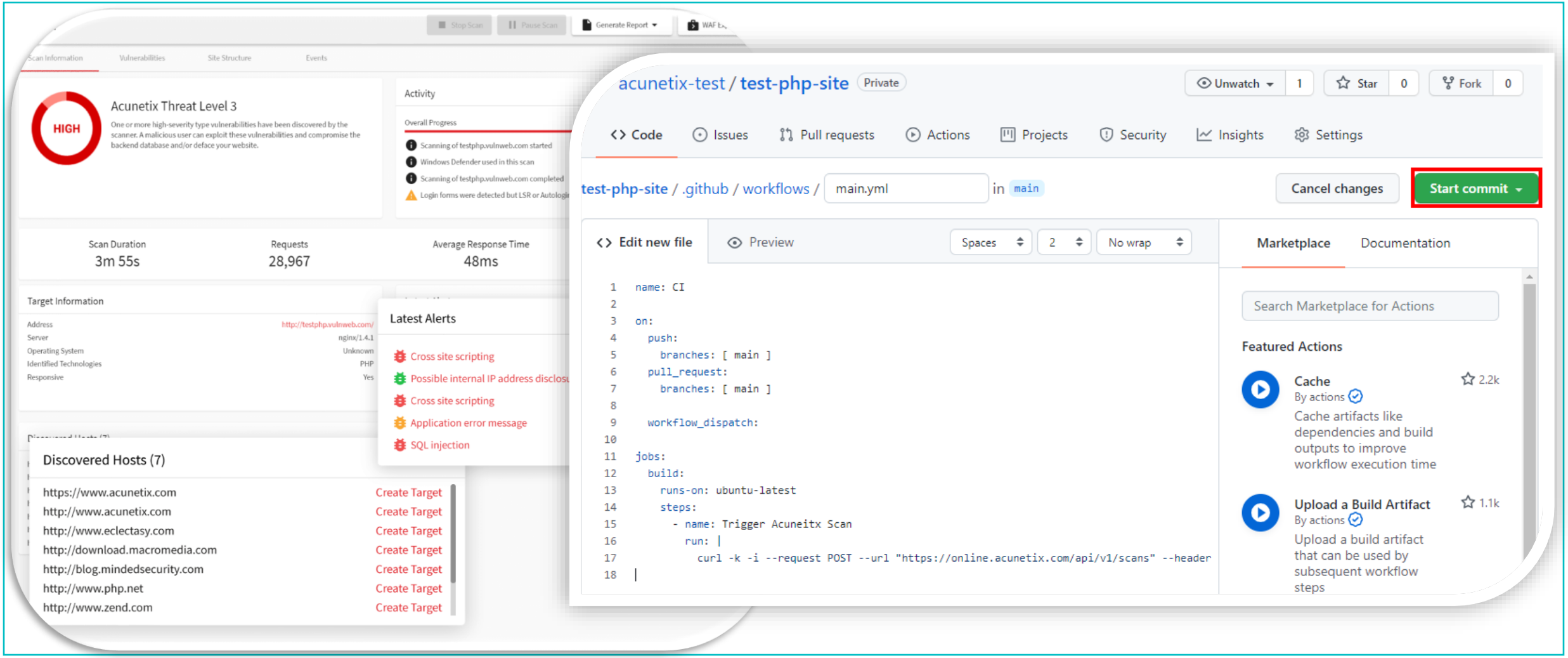


- O inventário de aplicações deve conter no mínimo: **Dono/Responsável,** Ambiente, Localização, **Criticidade**, **Classificação da Informação**
- O hacker ético irá atuar nas fases de realizar scan interno e/ou externo e obter/elaborar relatórios

MÉTODOS DE ANÁLISE

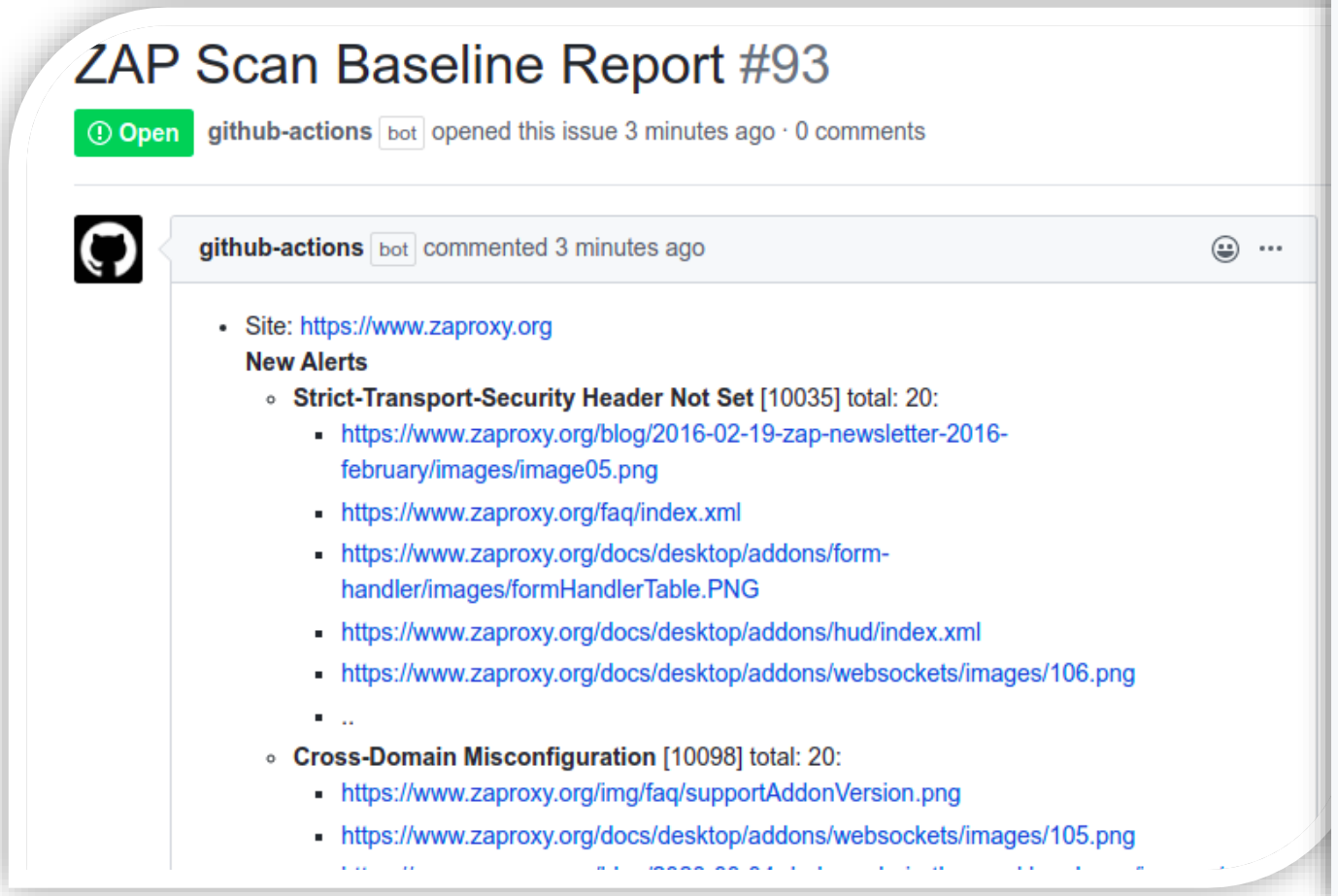
- Interno e Externo
- **Autenticado** e Não Autenticado
- Agent Based
- IAST
- Integrado ao Pipeline do SDLC (DAST)

MÉTODOS DE ANÁLISE



Fonte: https://www.acunetix.com/. **Integração do Acunetix com o Pipeline**

MÉTODOS DE ANÁLISE



ZAP Scan Baseline Report #93

Open github-actions bot opened this issue 3 minutes ago · 0 comments

github-actions bot commented 3 minutes ago

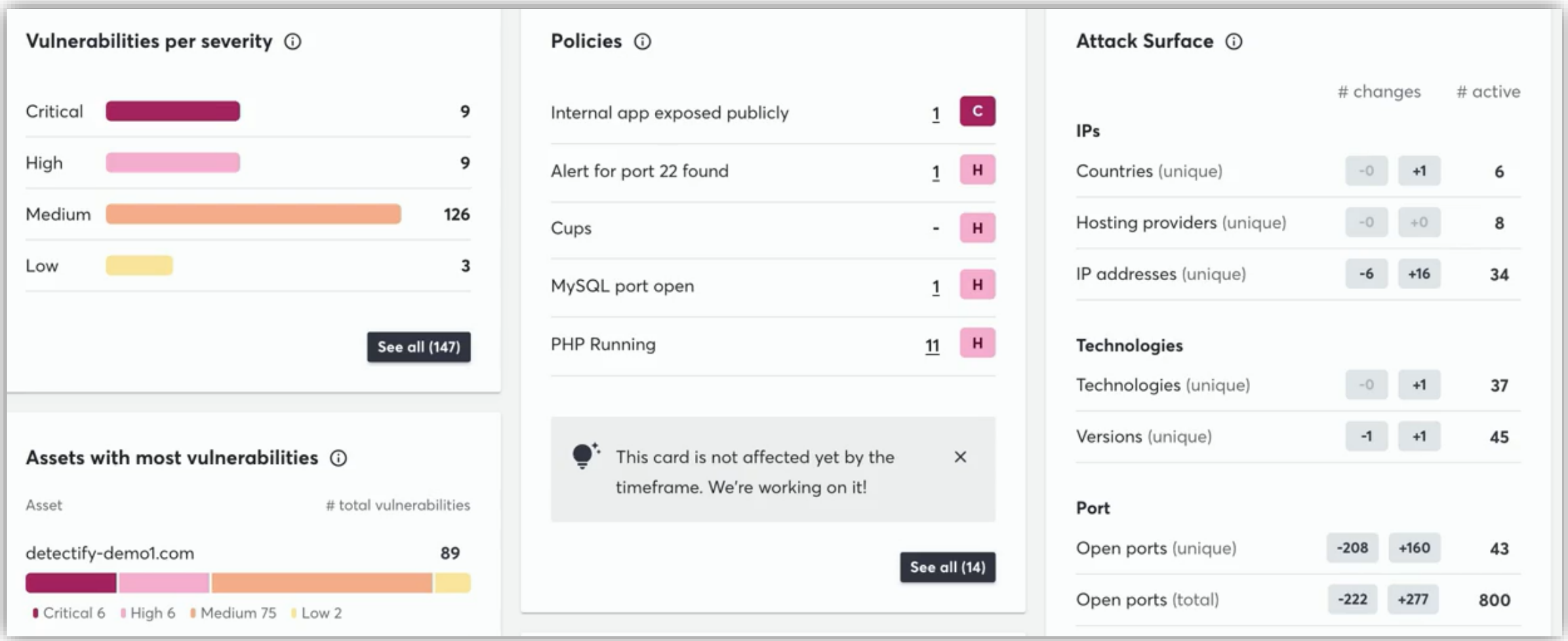
- Site: https://www.zaproxy.org
  New Alerts
  - **Strict-Transport-Security Header Not Set** [10035] total: 20:
    - https://www.zaproxy.org/blog/2016-02-19-zap-newsletter-2016-february/images/image05.png
    - https://www.zaproxy.org/faq/index.xml
    - https://www.zaproxy.org/docs/desktop/addons/form-handler/images/formHandlerTable.PNG
    - https://www.zaproxy.org/docs/desktop/addons/hud/index.xml
    - https://www.zaproxy.org/docs/desktop/addons/websockets/images/106.png
    - ..
  - **Cross-Domain Misconfiguration** [10098] total: 20:
    - https://www.zaproxy.org/img/faq/supportAddonVersion.png
    - https://www.zaproxy.org/docs/desktop/addons/websockets/images/105.png

```yaml
on: [push]

jobs:
  zap_scan:
    runs-on: ubuntu-latest
    name: Scan the webapplication
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          ref: master
      - name: ZAP Scan
        uses: zaproxy/action-full-scan@v0.12.0
        with:
          token: ${{ secrets.GITHUB_TOKEN }}
          docker_name: 'ghcr.io/zaproxy/zaproxy:stable'
          target: 'https://www.zaproxy.org/'
          rules_file_name: '.zap/rules.tsv'
          cmd_options: '-a'
```

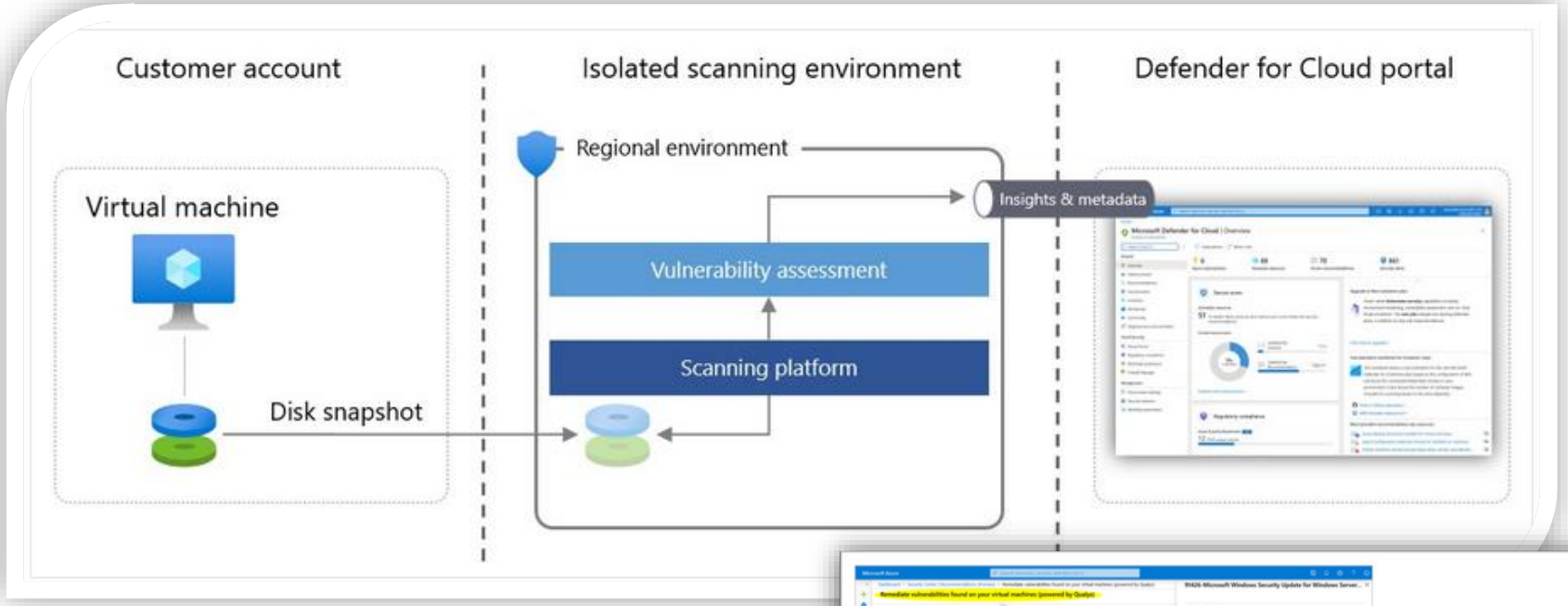
Fonte: https://www.zaproxy.org. **Integração do OWASP ZAP SCAN com o GitHub Actions**

MÉTODOS DE ANÁLISE

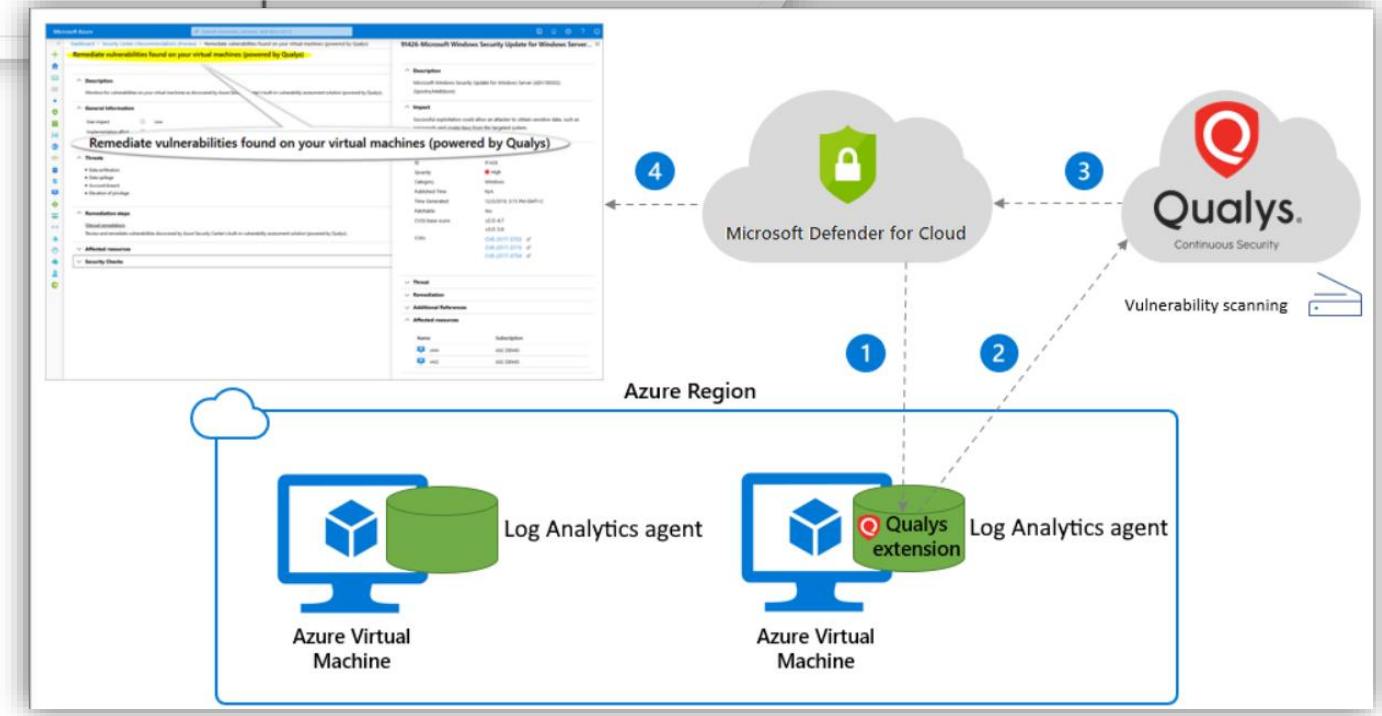


Fonte: https://detectify.com/. **Um serviço de scan externo para verificar vulnerabilidades continuamente no ambiente de Produção.**

PUC Minas



Análise de vulnerabilidades contínua em máquinas virtuais e ambiente de contêineres. Geralmente são utilizados métodos de agent based e agent less.



Referência: https://learn.microsoft.com. **Agent Less ou Agent Based** para análise de vulnerabilidades.
IMPORTANTE: Na figura de abaixo, o agente da Qualys não é mais utilizado no cenário do Microsoft Azure.

SOFTWARES


detectify


acunetix


Nessus®


BURPSUITE


OWASP ZAP




w3af
Web Application Attack and Audit Framework


Qualys®


RAPID7

PUC Minas

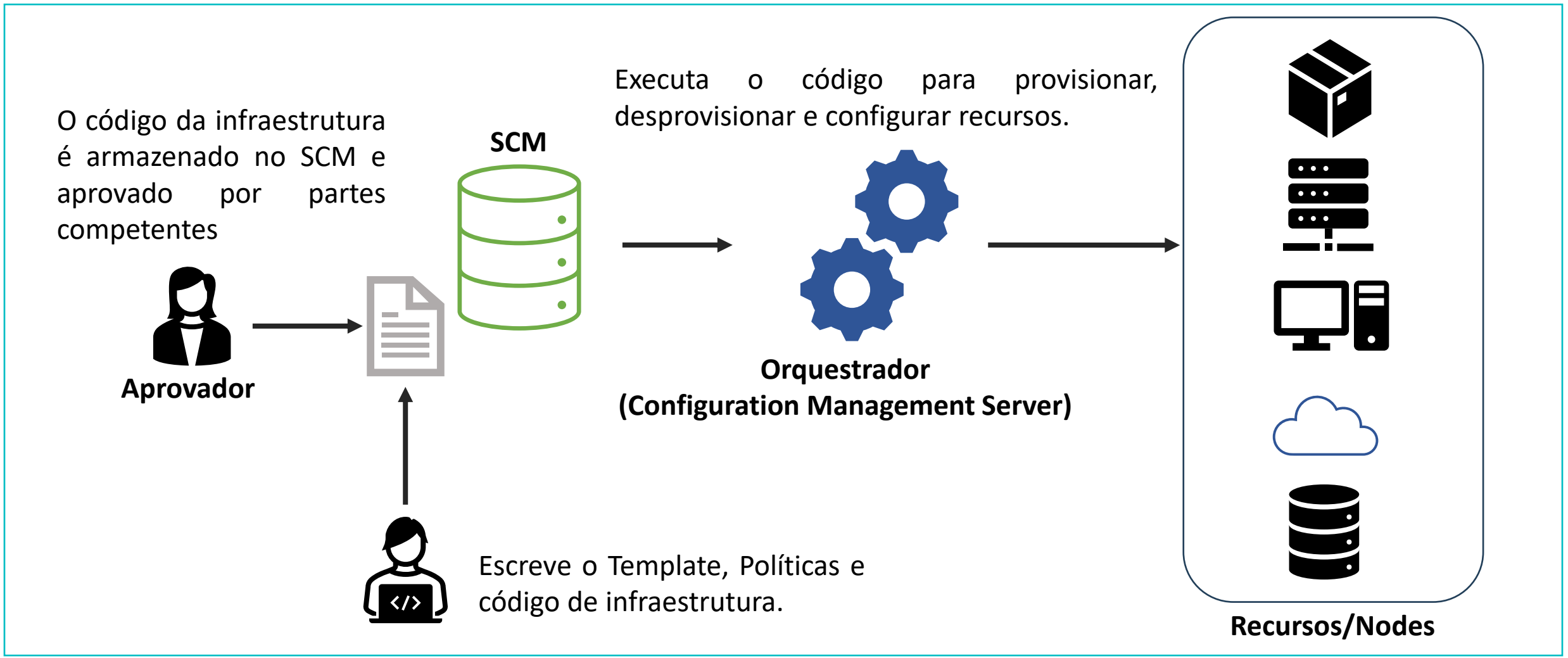
PUC Minas

INFRAESTRUTURA COMO CÓDIGO EM DEVSECOPS

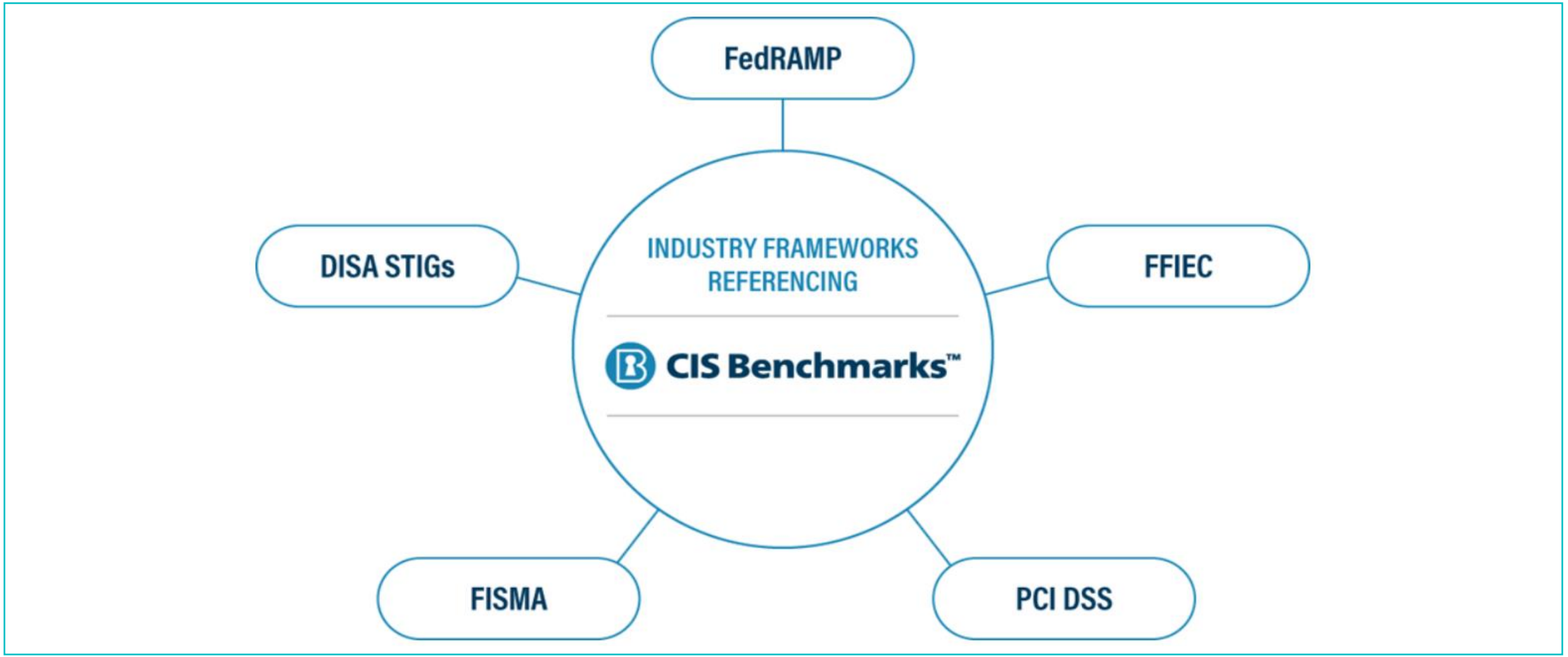
IAC

- Processo de automatização do gerenciamento, provisionamento e desprovisionamento da infraestrutura e seus elementos, como serviços, redes, máquinas virtuais, contêineres, balanceadores de carga usando código em vez de processos manuais
- Permite a criação e versionamento da infraestrutura, assim como a devida gestão de configuração do ambiente

IAC

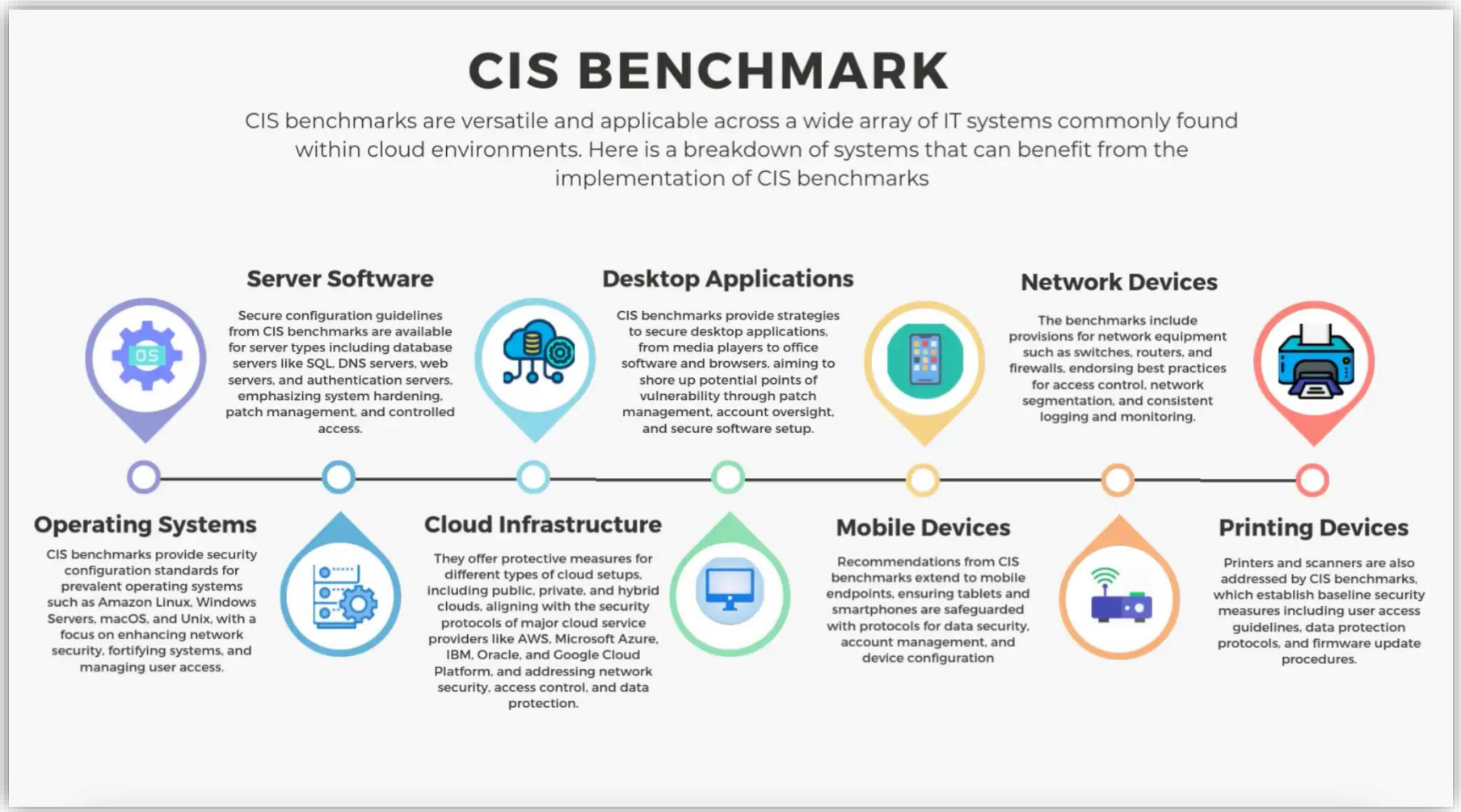

O código da infraestrutura é armazenado no SCM e aprovado por partes competentes
SCM
Executa o código para provisionar, desprovisionar e configurar recursos.
Aprovador
Orquestrador
(Configuration Management Server)
Escreve o Template, Políticas e código de infraestrutura.
Recursos/Nodes

IAC



Fonte: https://www.cisecurity.org

IAC



Fonte: https://www.paloaltonetworks.com

IAC

- O CIS Benchmark pode ser implantado como gestão de configuração por meio de infraestrutura como código (IaC)
- Imagens e instâncias/máquinas virtuais Linux/Windows de marketplace Azure, AWS e Google estão disponíveis já com o CIS Benchmark configurado
- A adoção do CIS Benchmark ajuda em manter conformidade com a ISO/IEC 27001, PCI-DSS, FISMA, entre outros
- A adoção do CIS Benchmark é usado para **Security Hardening**

IAC



Fonte: https://azuremarketplace.microsoft.com | https://aws.amazon.com

PUC Minas

PUC Minas

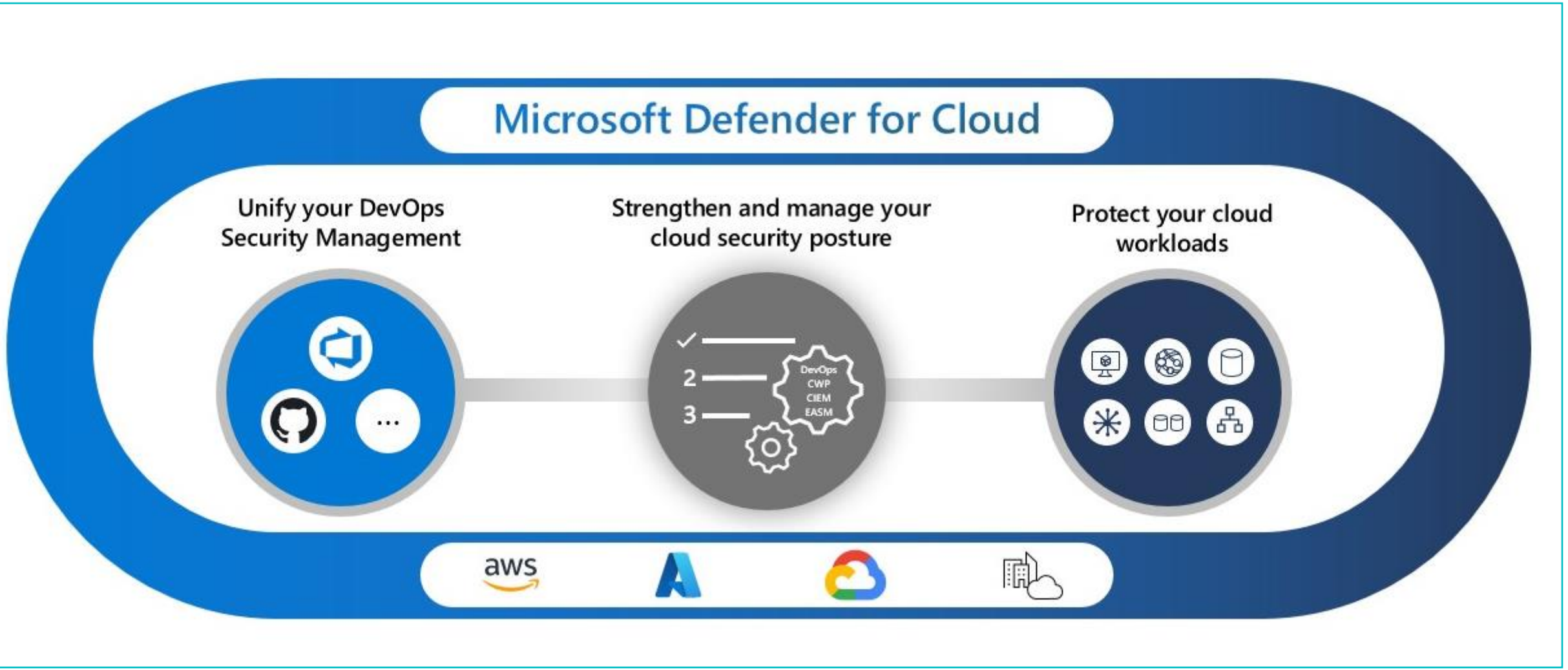
PUC Minas

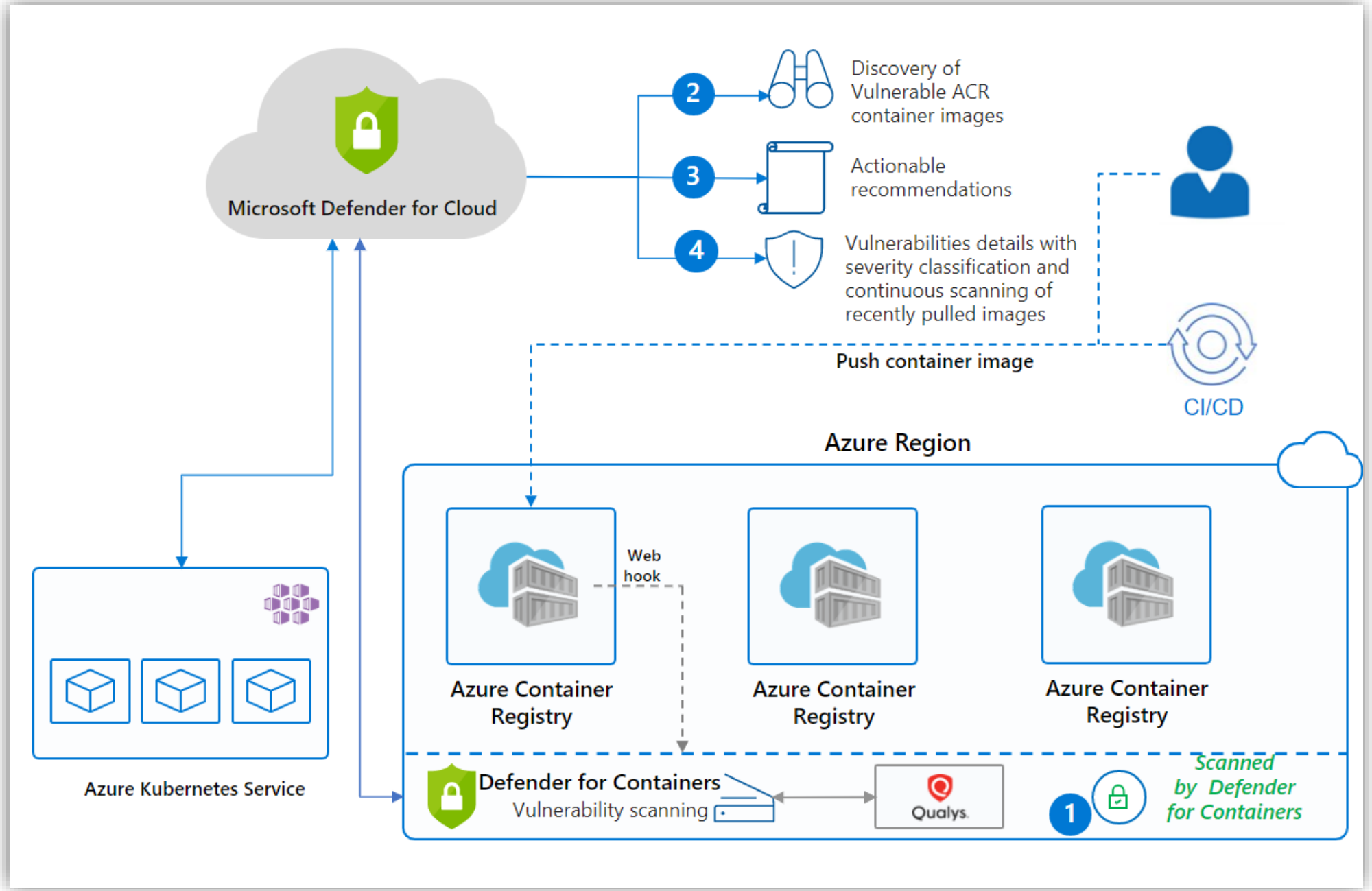
IMPLANTAÇÃO E SEGURANÇA DO AMBIENTE

SEGURANÇA DO AMBIENTE

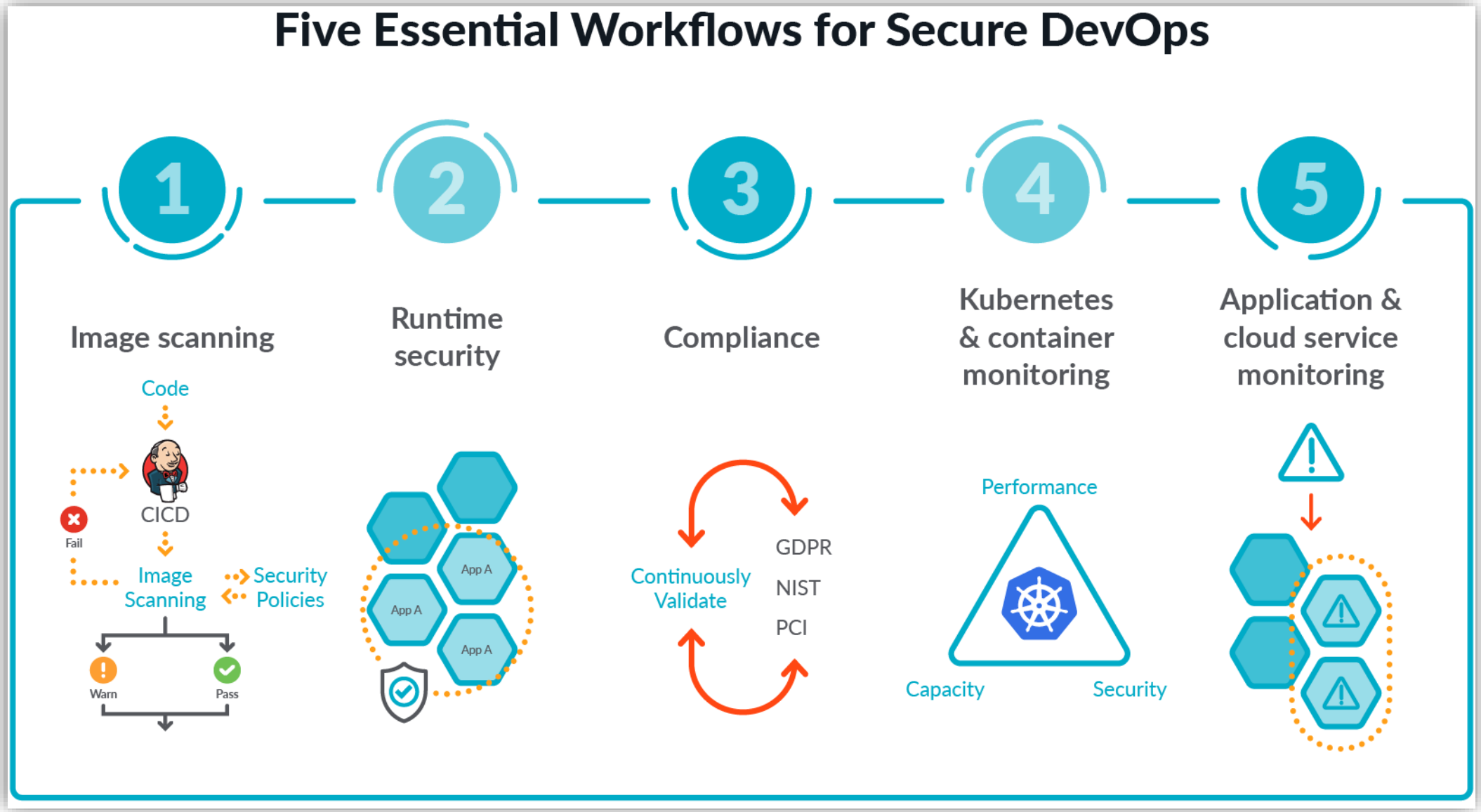
- O ambiente de Produção deve ser protegido enquanto a aplicação crítica de negócio está em execução
- Toda a carga de trabalho que sustenta a aplicação deve ser protegida
- Os ambientes devem ser segregados (Produção, Homologação, QA)
- Dados de Produção não devem ser utilizados em outros ambientes

SEGURANÇA DO AMBIENTE





Referência: https://Microsoft.com. Scan de Imagens de Contêineres (ACR).



Fonte: https://sysdig.com. Soluções nativas de Cloud e de **análise de vulnerabilidades** para **segurança de imagens de contêineres**

SEGURANÇA DO AMBIENTE



Exemplo de avaliação **Compliance** do ambiente de Produção com a ISO/IEC 27001:2022. O mesmo é aplicável para PCI-DSS, HIPAA, FISMA etc.

PUC Minas

MÉTODOS DE IMPLANTAÇÃO E LIBERAÇÃO

IMPLANTAÇÃO E LIBERAÇÃO

- **Implantação (Deployment)**
  - Refere-se ao **processo de implantação** por meios técnicos no ambiente
- **Liberação (Release)**
  - Refere-se a **disponibilizar o produto** e seus recursos após a implantação **para os clientes**

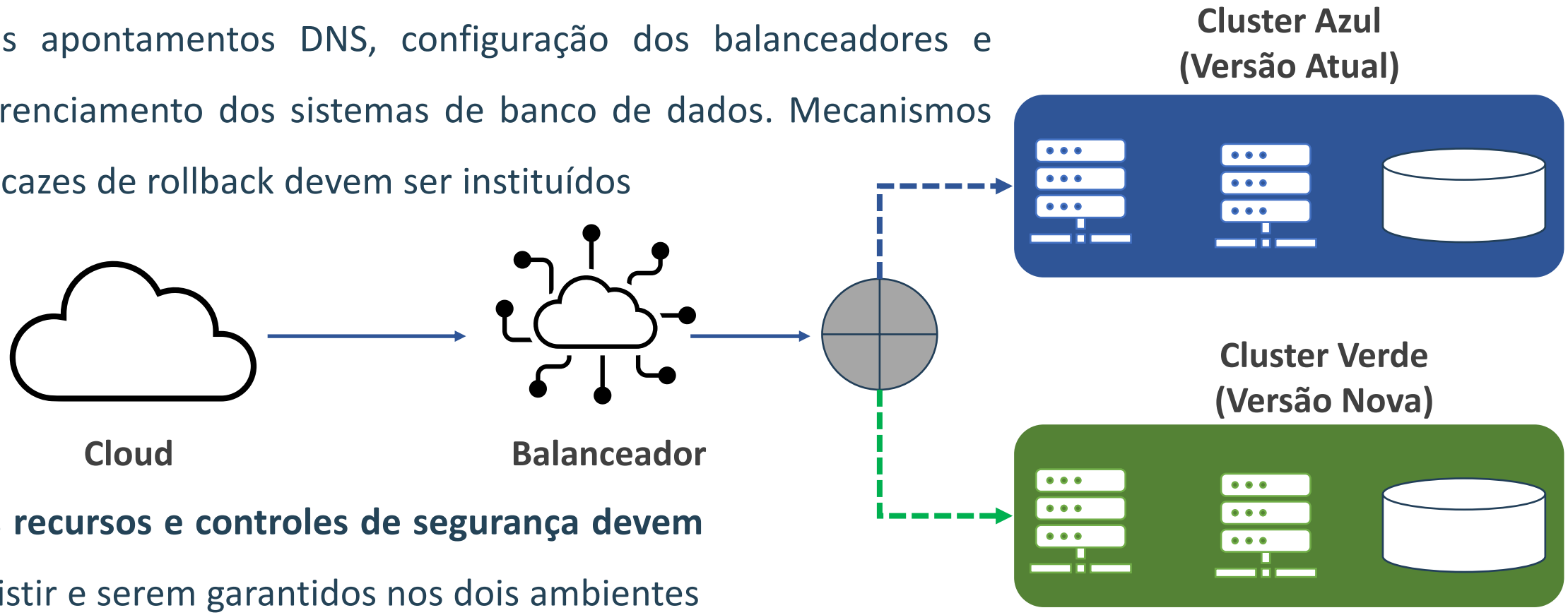
IMPLANTAÇÃO E LIBERAÇÃO

BASEADO NO AMBIENTE	BASEADO NO APLICATIVO
Há no mínimo dois ambientes, sendo que apenas um deles fica ativo para o cliente (requer configuração em balanceadores e outros recursos)	Uso de configurações (mais simples) na aplicação que não requerem o processo de implantação, por exemplo, habilitar um recurso a partir de uma aquisição, plugins etc.
Blue-Green, Staging e Canary.	Alternância de recursos (Feature Toggles, Bits de Recurso ou Flippers) e lançamento escuro (sem o usuário perceber)

O foco da nossa abordagem é baseado no ambiente.

IMPLANTAÇÃO E LIBERAÇÃO

Desafios na estratégia blue-green incluem a automatização dos apontamentos DNS, configuração dos balanceadores e gerenciamento dos sistemas de banco de dados. Mecanismos eficazes de rollback devem ser instituídos



Os **recursos e controles de segurança devem** existir e serem garantidos nos dois ambientes

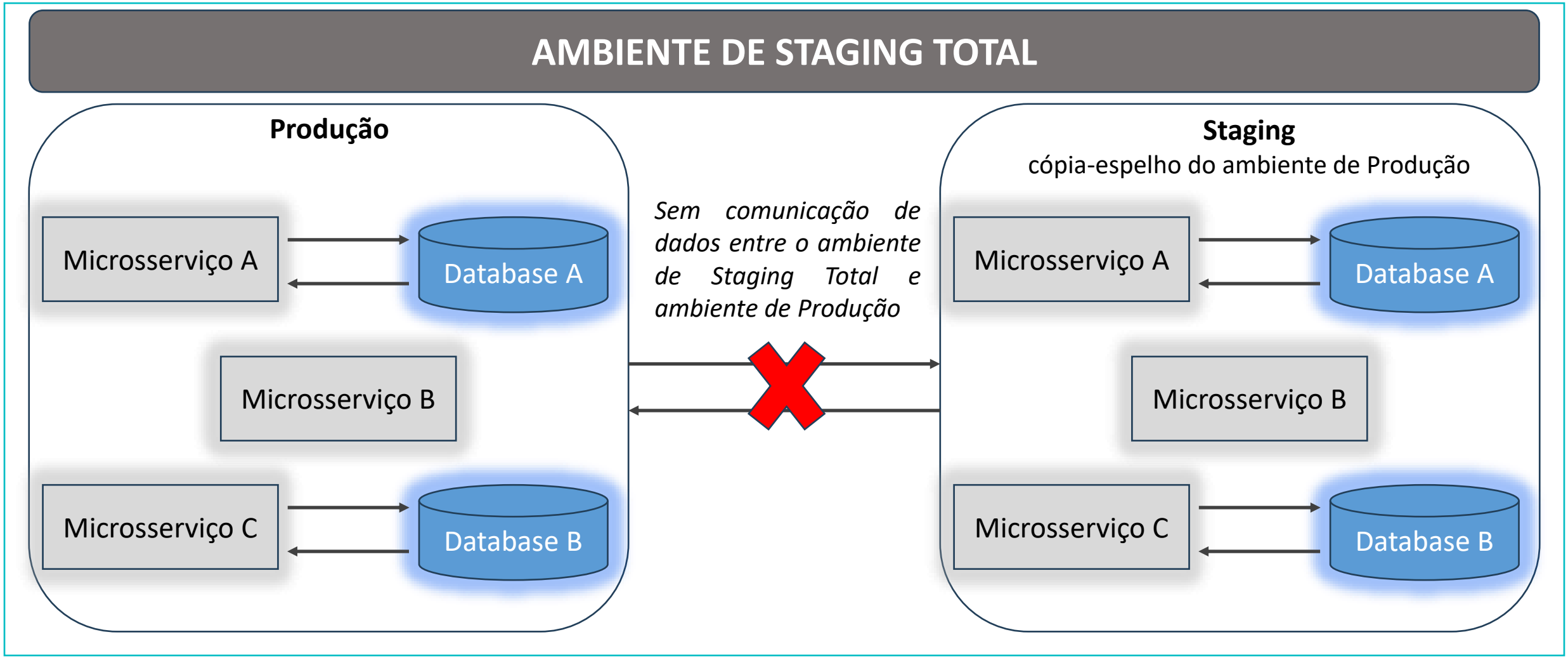
IMPLANTAÇÃO E LIBERAÇÃO

- **Ambiente de Staging**
  - Ambiente que **acomoda a versão candidata à produção**. Requer-se que seja um “espelho” do ambiente de Produção quanto à gestão da configuração, mas não necessariamente em escala
  - Diferente do ambiente de desenvolvimento, teste, homologação ou QA, pois trata-se de um ambiente com a versão testada e pronta para a produção (testes de software não ocorrem neste ambiente)

IMPLANTAÇÃO E LIBERAÇÃO

- **Ambiente de Pré-Release (Canary)**
  - Após a fase de Staging pode realizar o deploy em Canary em que entre 5% a 10% da carga de trabalho do ambiente de produção é utilizada
  - Em caso de sucesso no comportamento da aplicação, novas implantações são realizadas no restante da carga de trabalho

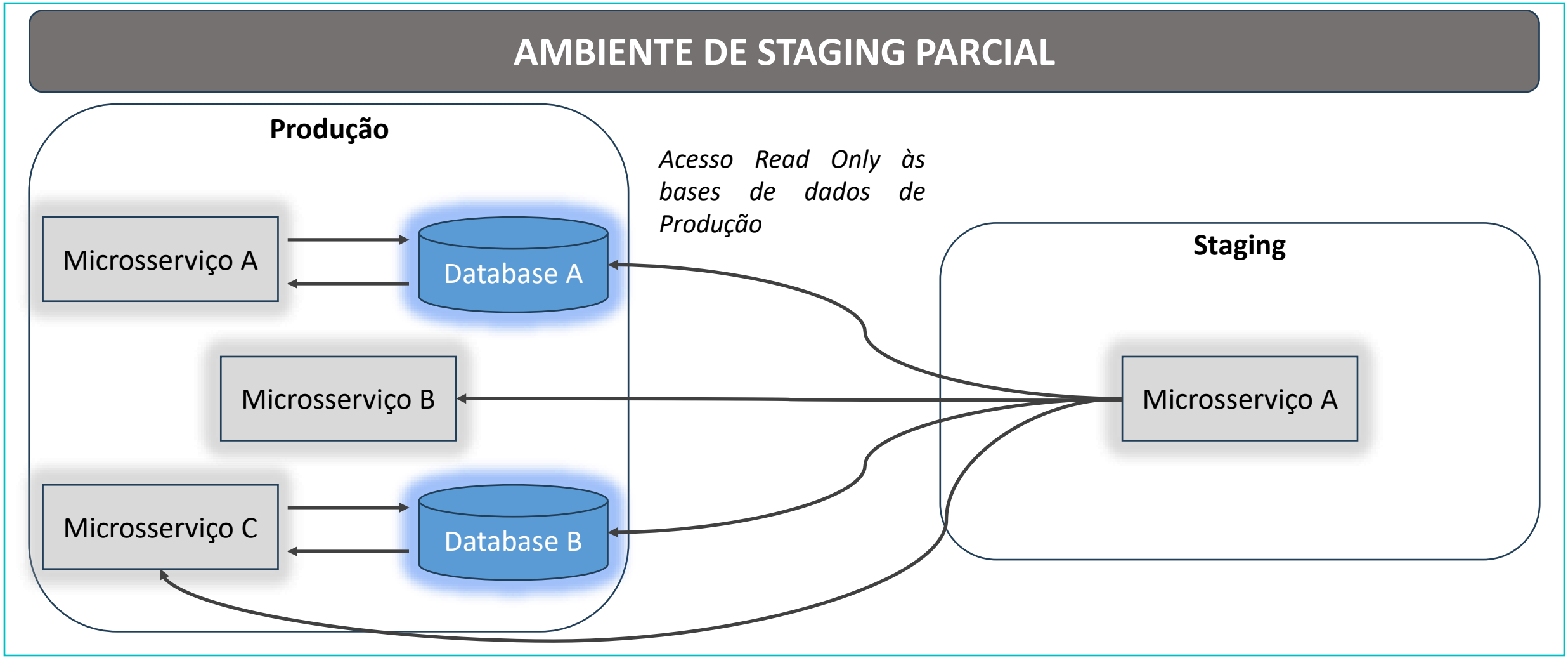
IMPLANTAÇÃO E LIBERAÇÃO



Fonte: Adaptado de: **FOWLER**. Susan J. **Microsserviços prontos para produção**. 2017. Novatec. São Paulo.

PUC Minas

IMPLANTAÇÃO E LIBERAÇÃO



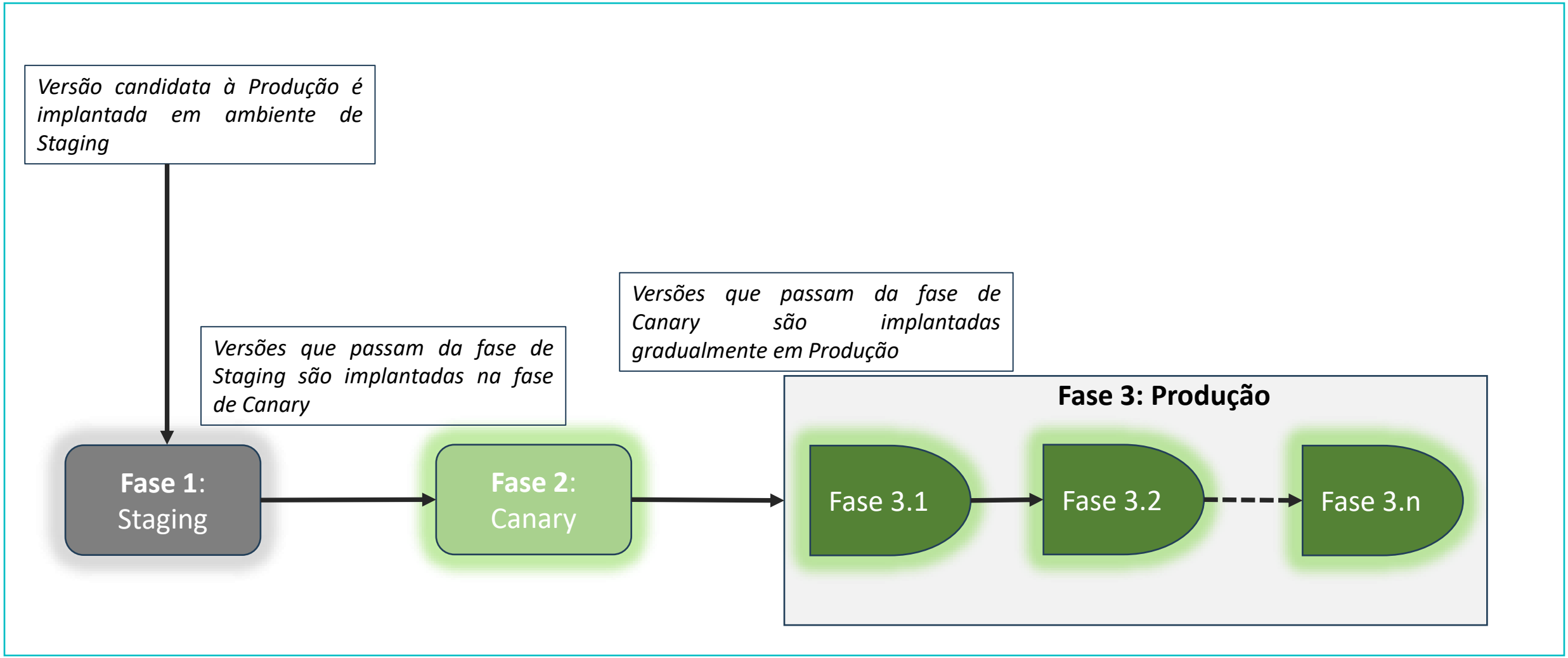
Fonte: Adaptado de: **FOWLER**. Susan J. **Microsserviços prontos para produção**. 2017. Novatec. São Paulo.

IMPLANTAÇÃO E LIBERAÇÃO

DIFERENÇAS ENTRE STAGING TOTAL E STAGING PARCIAL		
	Staging Total	Staging Parcial
Cópia completa do ambiente de Produção	Sim	Não
Portas de Staging de Backend e Frontend separadas	Sim	Sim
Acesso a serviços de Produção	Não	Sim
Acesso de leitura aos databases de Produção	Não	Sim
Acesso de escrita aos databases de Produção	Não	Sim
Requer rollback automatizado	Não	Sim

Fonte: Adaptado de: **FOWLER**. Susan J. **Microsserviços prontos para produção**. 2017. Novatec. São Paulo.

IMPLANTAÇÃO E LIBERAÇÃO



Fonte: Adaptado de: **FOWLER**. Susan J. **Microsserviços prontos para produção**. 2017. Novatec. São Paulo.

PUC Minas

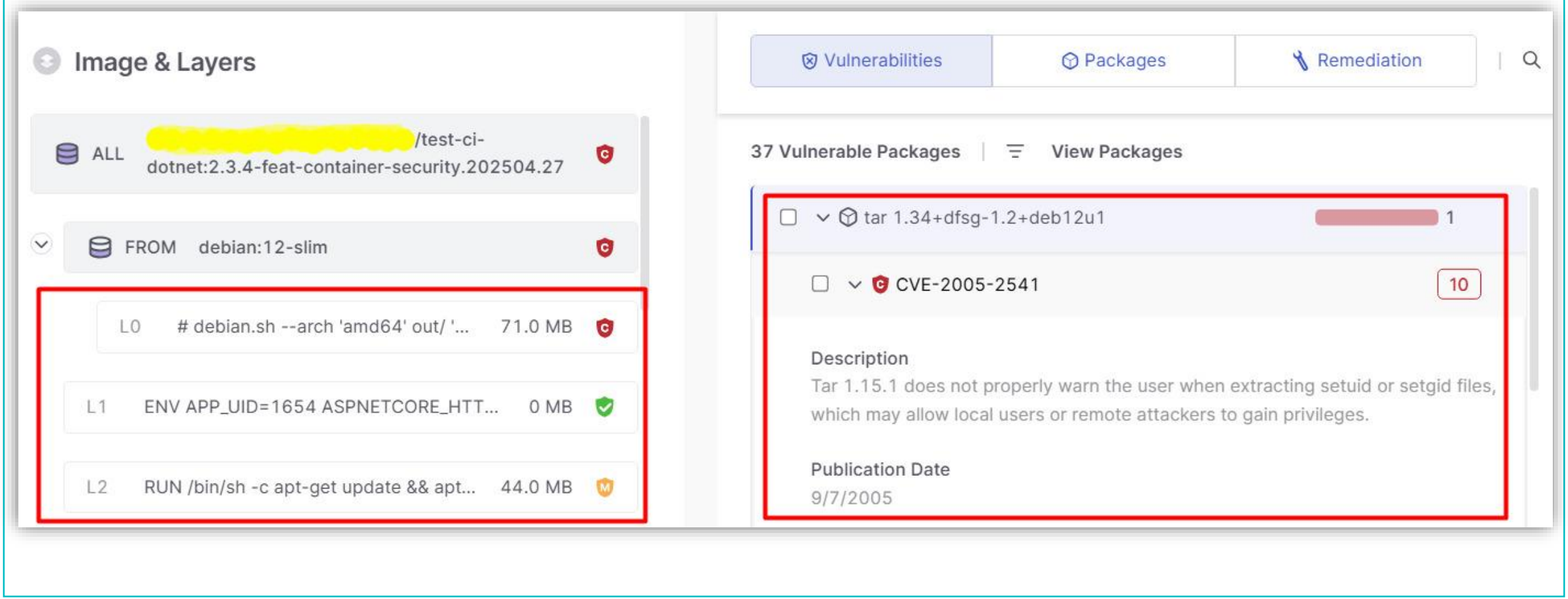
PUC Minas

SECURITY SDLC: OPERATE E MONITOR

OPERATE E MONITOR

- Nesta fase as equipes de DevSecOps devem manter a segurança do ambiente em produção
- Monitorar a infraestrutura, rede e aplicações
- Identificar ameaças de segurança e problemas de compliance
- Prover e revisar feedback

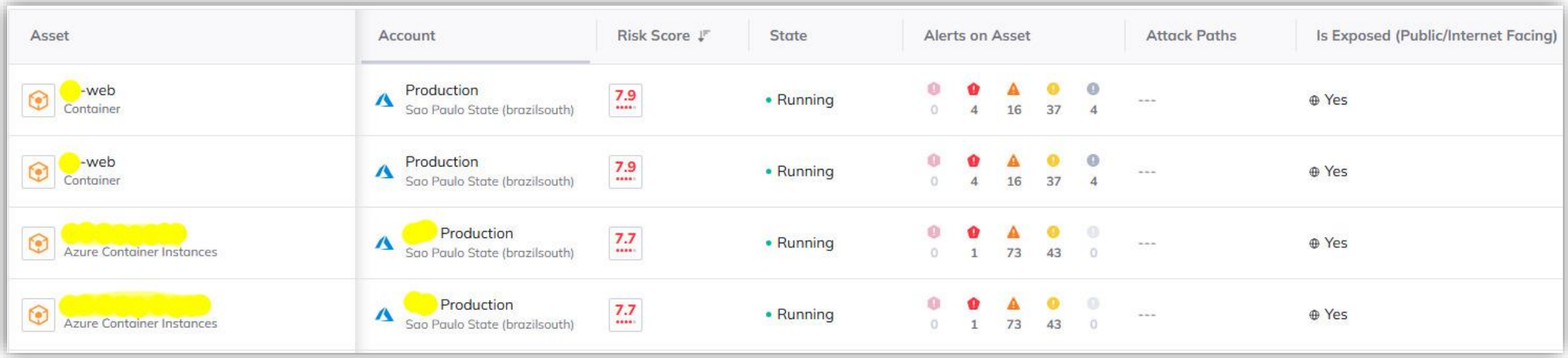
OPERATE E MONITOR



Encontrando **vulnerabilidades em contêineres por meio de Layers**.

PUC Minas

OPERATE E MONITOR

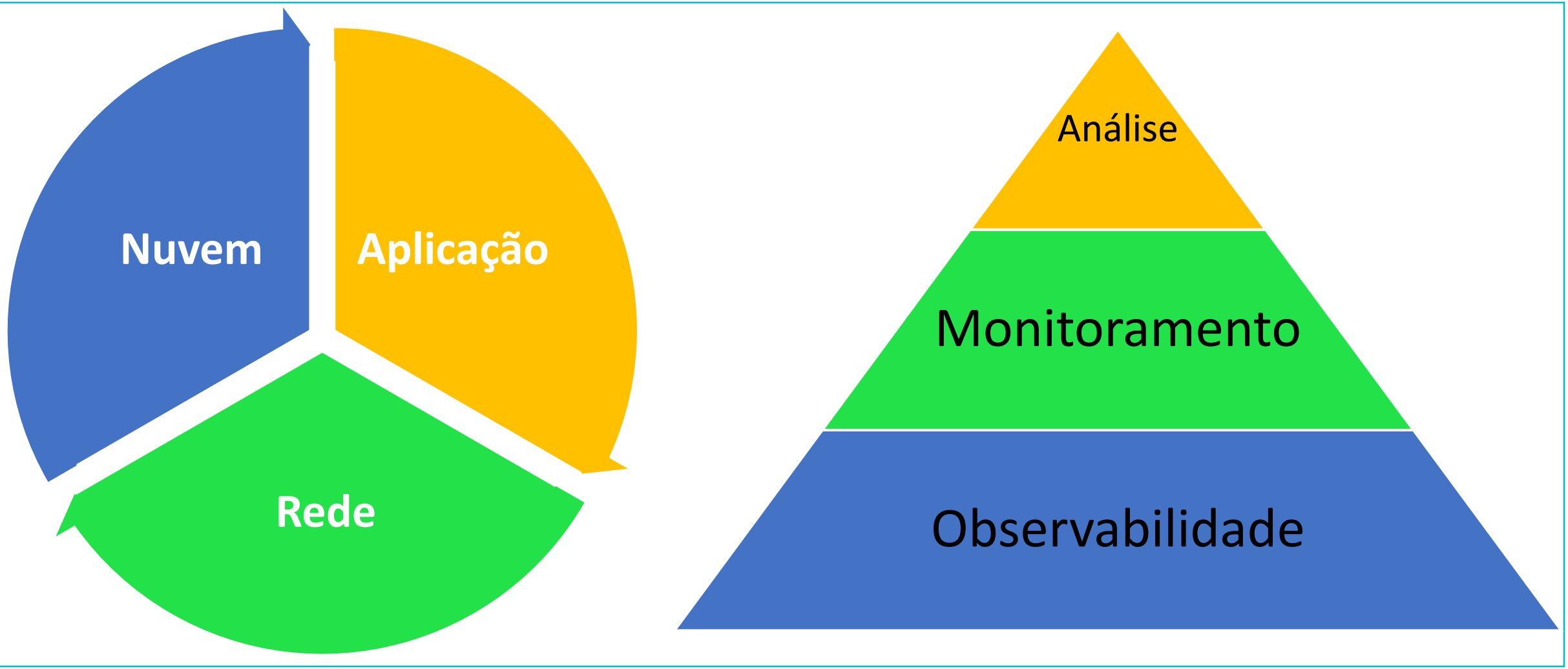


Asset	Account	Risk Score	State	Alerts on Asset	Attack Paths	Is Exposed (Public/Internet Facing)
[redacted]-web Container	Production Sao Paulo State (brazilsouth)	7.9	Running	0 4 16 37 4	---	Yes
[redacted]-web Container	Production Sao Paulo State (brazilsouth)	7.9	Running	0 4 16 37 4	---	Yes
[redacted] Azure Container Instances	[redacted]Production Sao Paulo State (brazilsouth)	7.7	Running	0 1 73 43 0	---	Yes
[redacted] Azure Container Instances	[redacted]Production Sao Paulo State (brazilsouth)	7.7	Running	0 1 73 43 0	---	Yes

Monitorando **vulnerabilidades em contêineres em execução**.

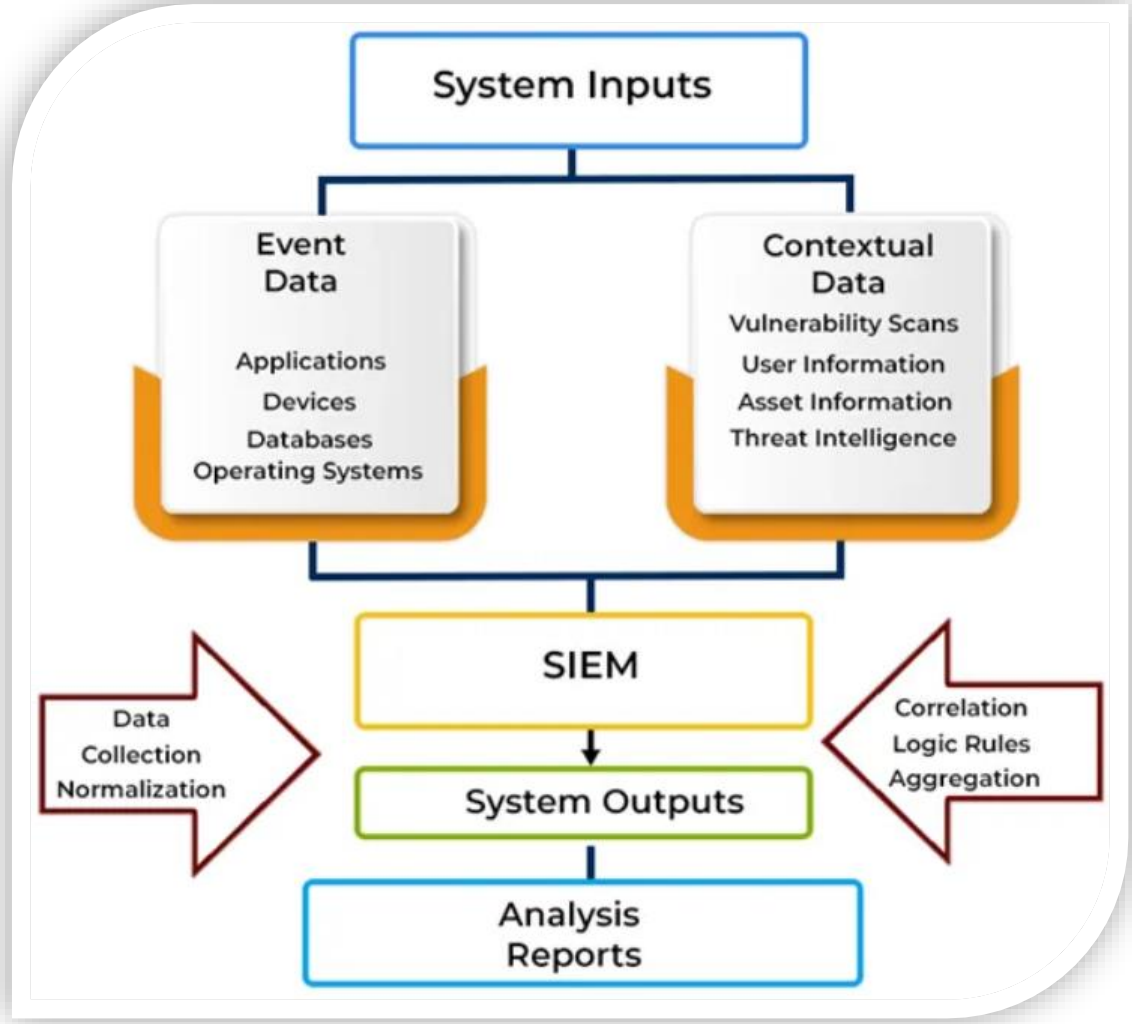
PUC Minas

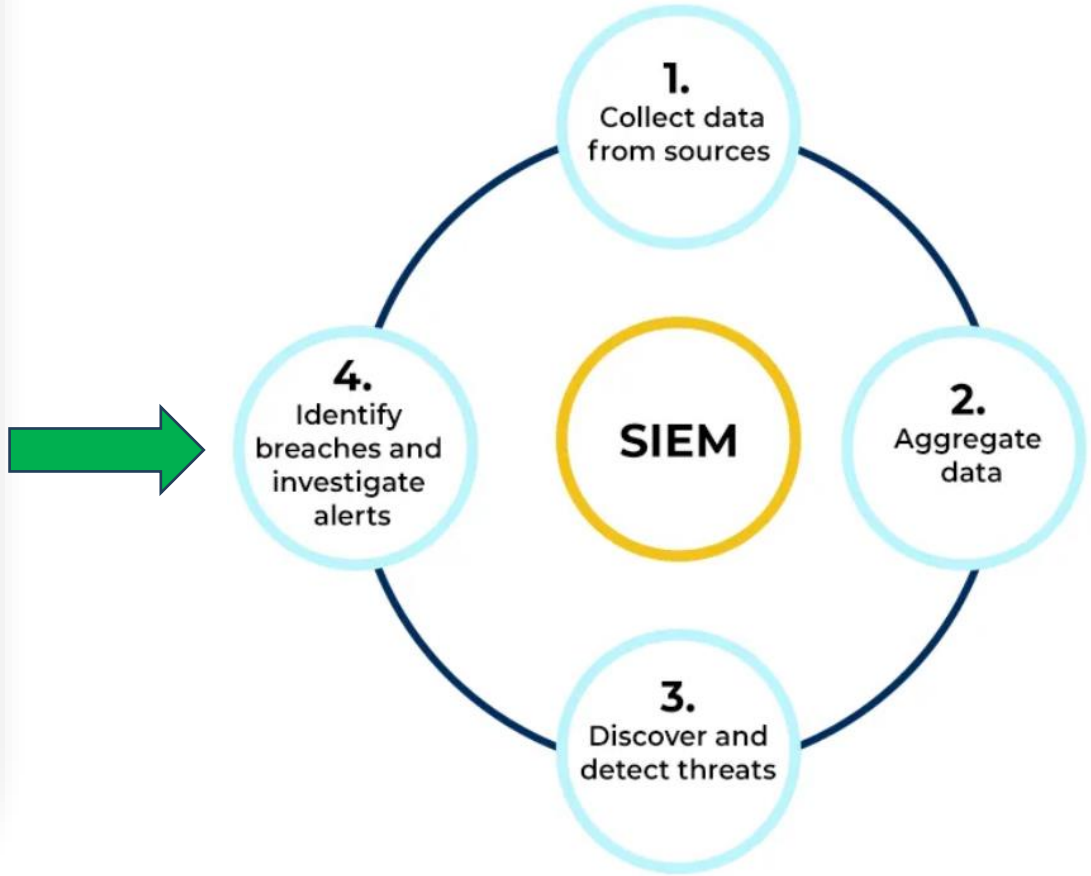
OPERATE E MONITOR



Dimensões do monitoramento e pirâmide do monitoramento e observabilidade.

OPERATE E MONITOR





Captura de dados, processamento e análise para gerar informação.

AGRADECIMENTO

Muito obrigado pela parceira e apoio!!!

Espero que esta disciplina tenha agregado valor para você!

Sucesso!!!

PUC Minas